

第 10 讲 并发与竞态

一、 并发和竞态

➤ 什么是并发？

- 多个执行单元同时、并行被执行。
- 竞态(race conditions)
 - 并发的执行单元对共享资源的访问则很容易导致。
 - 共享资源:
硬件资源，软件上的全局变量、静态变量等

➤ Linux 内核中，什么情况会发生竞态？

- 对称多处理器(SMP)的多个 CPU
- 单 CPU 内进程与抢占它的进程
- 中断(硬中断、软中断、Tasklet、底半部)与进程之间

➤ 如何解决竞态问题？

- 保证对共享资源的互斥访问，
指一个执行单元在访问共享资源的时候，其他的执行单元被禁止访问。
- Linux 设备驱动中可采用的互斥途径
中断屏蔽；原子操作；自旋锁；信号量；completion

二、 中断屏蔽

➤ 中断屏蔽：

- 可解决中断与进程之间的并发；
- 也可解决内核抢占进程之间的并发
- 主要函数

```
#include <linux/irqflags.h>
local_irq_disable()      //禁止中断
local_irq_enable()       //打开中断
local_irq_save(flags)    //禁止中断并保存中断寄存器信息到 flags
local_irq_restore(flags) //打开中断并恢复 flags 值到中断寄存器
local_bh_disable()       //仅禁止中断底半部中断
local_bh_enable()        //打开中断底半部中断
local_irq_disable()//屏蔽中断
...
critical section //临界区
...
local_irq_enable()//打开中断
```

注意：不要长时间屏蔽中断，由于 Linux 系统的异步 I/O、进程调度等很多重要操作都依赖于中断，在屏蔽中断期间所有的中断都无法得到处理，因此长时间屏蔽中断是很危险的，有可能造成数据丢失甚至系统崩溃

三、 信号量和互斥体

➤ 信号量（ semaphore ）

- 信号量本质上是一个整数值，
- 一对操作函数通常称为 P 和 V。
- 进入临界区
 - 相关信号量上调用 P；
 - 如果信号量 > 零，则该值会减小一，而进程可以继续。
 - 如果信号量的值 = 零(或更小)，进程必须等待直到其他人释放该信号量。
- 退出临界区
 - 信号量的解锁通过调用 V 完成；
 - 该函数增加信号量的值，
 - 并在必要时唤醒等待的进程。

➤ 当信号量用于互斥

- 避免多个进程同时在一个临界区中运行
- 信号量的值应初始化为 1。
 - 只能由单个进程或线程拥有。
 - 一个信号量有时也称为一个“互斥体(mutex)”，它是互斥(mutual exclusion)的简称。
 - Linux 内核中几乎所有的信号量均用于互斥。

➤ 初始化信号量

- 信号量类型为 struct semaphore；
- 信号量可通过几种途径来声明和初始化：
 - 动态的初始化信号量

```
#include <linux/semaphore.h>

void sema_init(struct semaphore *sem, int val); //val 是信号量的初始值
```
 - 静态声明互斥信号量：

```
#define DECLARE_MUTEX(name) \    //声明互斥信号量“name”，并初始化为 1
    struct semaphore name = __SEMAPHORE_INITIALIZER(name, 1)
```
 - 动态的初始化互斥信号量：

```
#define init_MUTEX(sem)          sema_init(sem, 1)
```

➤ 获得信号量

void down(struct semaphore *sem); //减小信号量的值，如果不能获得信号量就一直等待
int down_interruptible(struct semaphore *sem); //完成和 down 相同的工作，但操作是可中断的
int down_trylock(struct semaphore *sem); //永远不会休眠，如果信号量在调用时不可获得，就会立即返回一个非零值

➤ 释放信号量

void up(struct semaphore *sem);

➤ 使用信号量模板

➤ 使用信号量模板

```
DECLARE_MUTEX(sem);
```

```
if(down_interruptible(&sem))
```

```
{
```

```
    return -ERESTARTSYS;
```

```
}
```

```
...
```

```
critical section
```

```
...
```

```
up(&sem);
```

定义信号量

获取信号量，保护临界区

临界区

释放信号量

➤ 读写信号量

➤ 读取者/写入者信号量

❑ 数据类型: `struct rw_semaphore`;

❑ 初始化:

```
void init_rwsem(struct rw_semaphore *sem);
```

❑ 主要函数:

```
#include <linux/rwsem.h>
void down_read(struct rw_semaphore *sem);
int down_read_trylock(struct rw_semaphore *sem);
void up_read(struct rw_semaphore *sem);

void down_write(struct rw_semaphore *sem);
int down_write_trylock(struct rw_semaphore *sem);
void up_write(struct rw_semaphore *sem);
void downgrade_write(struct rw_semaphore *sem);
```

➤ 读取者/写入者信号量实例

```
rw_semaphore_t rw_sem;
```

```
init_rwsem(&rw_sem);
```

```
//读时获取信号量
```

```
down_read(&rw_sem);
```

```
... //临界资源
```

```
up_read(&rw_sem);
```

```
down_write(&rw_sem);
```

```
... //临界资源
```

```
up_write(&rw_sem);
```

定义读写信号量

初始化读写信号量

获取临界资源

临界区

释放临界资源

四、 Completion

一种轻量级的机制

它允许一个线程告诉另一线程某个工作已经完成。

➤ 初始化

```
#include <linux/completion.h>
DECLARE_COMPLETION(xxx_completion);
struct completion xxx_completion;
init_completion(&xxx_completion);
```

➤ 等待 completion

```
/* kernel/sched.c */
void wait_for_completion(struct completion *c);
```

➤ 触发完成

```
/* kernel/sched.c */
void complete(struct completion *c);
void complete_all(struct completion *c); //必须在重复使用该结构之前重新初始化它。下面这个宏可用来快速执行重新初始化: INIT_COMPLETION(struct completion c);
```

➤ completion使用示例

```
DECLARE_COMPLETION(xxx_comp);  
ssize_t complete_read(struct file *filp,  
                        char __user *buf, size_t count,  
                        loff_t *pos)  
{  
    wait_for_completion(&xxx_comp);  
    return 0;  
}  
  
ssize_t complete_write(struct file *filp,  
                        const char __user *buf, size_t count,  
                        loff_t *pos)  
{  
    complete(&xxx_comp);  
    return count;  
}
```

定义并初始化completion

等待某个事件完成

表示某个事件完成

五、 自旋锁

➤ 概念

- 一个自旋锁是一个互斥设备,
- 只有两个值: “**锁定**”和“**解锁**”。
 - 实现为某个整数值中的单个位。希望获得某特定锁的代码测试相关的位。
 - 锁可用, 则“**锁定**”位被设置, 而代码继续进入临界区
 - 如果锁被其他人获得, 则代码进入忙循环并重复检查这个锁, 直到该锁可用为止。这个循环就是自旋锁的“**自旋**”

➤ 初始化自旋锁

```
#include <linux/spinlock.h>  
spinlock_t xxx_lock=SPIN_LOCK_UNLOCKED;  
void spin_lock_init(spinlock_t *lock);
```

➤ 锁定函数

```
void spin_lock(spinlock_t *lock);  
void spin_lock_irqsave(spinlock_t *lock, unsigned long flags);  
void spin_lock_irq(spinlock_t *lock);  
void spin_lock_bh(spinlock_t *lock);
```

➤ 释放自旋锁

```
void spin_unlock(spinlock_t *lock);  
void spin_unlock_irqrestore(spinlock_t *lock, unsigned long flags);  
void spin_unlock_irq(spinlock_t *lock);  
void spin_unlock_bh(spinlock_t *lock);
```

➤ 自旋锁使用实例

`spinlock_t lock;`

定义一个自旋锁

`spin_lock_init(&lock);`

初始化自旋锁

`spin_lock(&lock);`

获取自旋锁，保护临界区

`critical section`

临界区

`spin_unlock(&lock);`

释放自旋锁

➤ 读写自旋锁

- ❑ 任意数量的读取者可以同时进入临界区
- ❑ 写入者必须互斥访问
- ❑ 变量类型: `rwlock_t`

➤ 初始化

```
#include <linux/spinlock.h>
rwlock_t xxx_rwlock = RW_LOCK_UNLOCKED;
rwlock_t xxx_rwlock;
rwlock_init(&xxx_rwlock);
```

➤ 读取者获得锁

```
void read_lock(rwlock_t *lock);
void read_lock_irqsave(rwlock_t *lock, unsigned long flags);
void read_lock_irq(rwlock_t *lock);
void read_lock_bh(rwlock_t *lock);
```

➤ 读取者释放锁

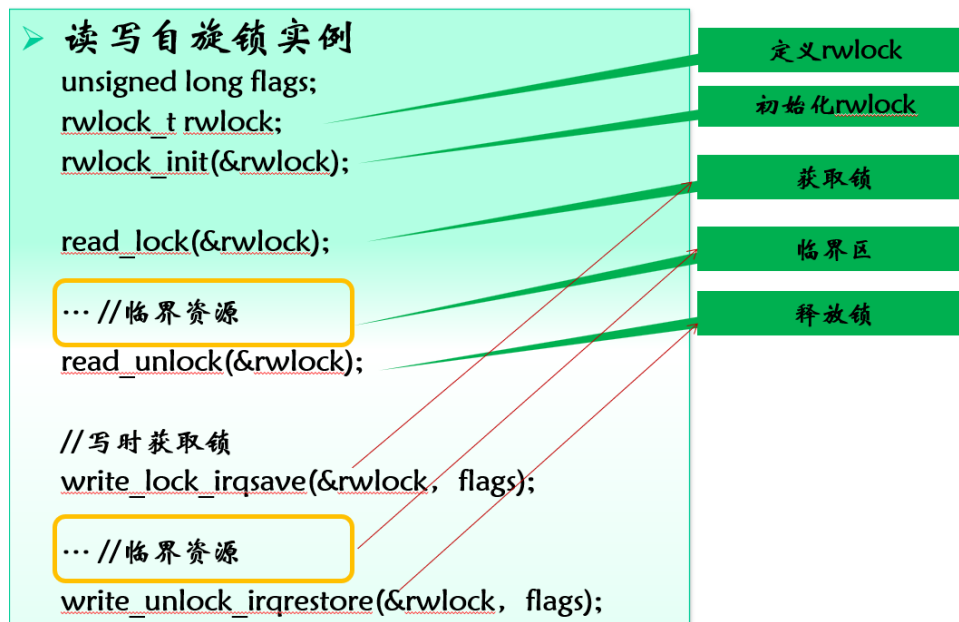
```
void read_unlock(rwlock_t *lock);
void read_unlock_irqrestore(rwlock_t *lock, unsigned long flags);
void read_unlock_irq(rwlock_t *lock);
void read_unlock_bh(rwlock_t *lock);
```

➤ 写入者获得锁

```
void write_lock(rwlock_t *lock);
void write_lock_irqsave(rwlock_t *lock, unsigned long flags);
void write_lock_irq(rwlock_t *lock);
void write_lock_bh(rwlock_t *lock);
int write_trylock(rwlock_t *lock);
```

➤ 写入者释放锁

```
void write_unlock(rwlock_t *lock);
void write_unlock_irqrestore(rwlock_t *lock, unsigned long flags);
void write_unlock_irq(rwlock_t *lock);
void write_unlock_bh(rwlock_t *lock);
```



➤ 自旋锁 VS 信号量

❑ 开销成本

- 使用信号量的开销是进程上下文切换时间
- 自旋锁的开销是忙等待获取自旋锁

❑ 等待机制不同

- 信号量可能导致阻塞，所以在不允许阻塞的代码中不能用可能引起阻塞的信号量处理方式
- 自旋锁是忙等待
- 比如，中断处理程序

➤ 自旋锁避免死锁规则

➤ 自旋锁的使用规则

- ❑ 任何拥有自旋锁的代码都必须是原子的；
- ❑ 如果中断处理函数中也要获得自选锁，那么驱动程序需要在拥有自旋锁时禁止中断；
- ❑ 自旋锁必须在可能的最短时间内拥有；

➤ 其他规则

- ❑ 避免某个获得锁的函数调用其他同样试图获取这个锁的函数，否则代码就会死锁；
- ❑ 不论是信号量还是自旋锁，都不允许锁拥有者第二次获得这个锁，如果试图这么做，系统将挂起；

➤ 锁的顺序规则

- ❑ 按同样的顺序获得锁
- ❑ 如果必须获得一个局部锁和一个属于内核更中心位置的锁，则应该首先获取自己的局部锁
- ❑ 如果我们拥有信号量和自旋锁的组合，则必须首先获得信号量；在拥有自旋锁时调用 down(可导致

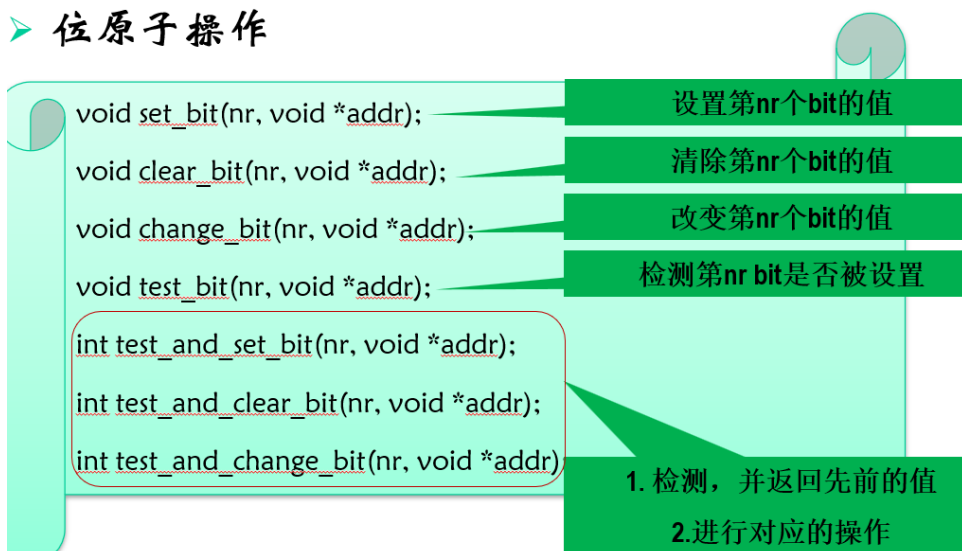
休眠)是个严重的错误的

六、 原子操作

➤ 整型原子操作

- ❑ 类型: atomic_t (#include <asm-generic/atomic.h>)
void atomic_set(atomic_t *v, int i);
atomic_t v = ATOMIC_INIT(0);
- ❑ 获取原子变量的值
int atomic_read(atomic_t *v);
- ❑ 原子变量加/减
void atomic_add(int i, atomic_t *v);
void atomic_sub(int i, atomic_t *v);
- ❑ 原子变量自增/自减
void atomic_inc(atomic_t *v);
void atomic_dec(atomic_t *v);
- ❑ 操作并测试
int atomic_inc_and_test(atomic_t *v);
int atomic_dec_and_test(atomic_t *v);
int atomic_sub_and_test(int i, atomic_t *v);
- ❑ 操作并返回
int atomic_add_return(int i, atomic_t *v);
int atomic_sub_return(int i, atomic_t *v);
int atomic_inc_return(atomic_t *v);
int atomic_dec_return(atomic_t *v);

➤ 位原子操作



七、 Seqlock

顺序锁(seqlock)是对读写锁的一种优化,提高了读锁和写锁的独立性。写锁不会被读锁阻塞,读锁也不会被写锁阻塞。写锁会被写锁阻塞。

用于能够区分读与写的场合, 并且是读操作很多、写操作很少, 写操作的优先权大于读操作。

被保护的共享资源中不能含有指针,因为写执行单元可能会使指针失效,当读执行单元正要访问该指针时,系统就会崩溃。

seqlock 的实现思路是, 用一个递增的整型数表示 sequence。

写操作进入临界区时, sequence++; 退出临界区时, sequence 再++。

写操作还需要获得一个锁(比如 mutex), 这个锁仅用于写写互斥,以保证同一时间最多只有一个正在进行的写操作。

当 sequence 为奇数时,表示有写操作正在进行,这时读操作要进入临界区需要等待,直到 sequence 变为偶数。

读操作进入临界区时,需要记录下当前 sequence 的值,等它退出临界区的时候用记录的 sequence 与当前 sequence 做比较, 不相等则表示在读操作进入临界区期间发生了写操作,这时候读操作读到的东西是无效的,需要返回重试。

➤ 顺序锁的初始化

```
seqlock_init(x);           //动态初始化
#define SEQLOCK(x)         //静态初始化
```

➤ 读操作

```
unsigned int read_seqbegin(const seqlock_t* sl);
int read_seqretry(const seqlock_t* sl, unsigned seq);
```

➤ 写操作

```
void write_seqlock( seqlock_t* sl);
void write_sequnlock(seqlock_t* sl);
```

➤ 顺序锁的写执行单元模式

```
write_seqlock(&lock);
..... //写操作代码
write_sequnlock(&lock);
```

➤ 顺序锁的读执行单元模式

```
unsigned int seq_num = 0;
do
{
    seq_num = read_seqbegin(&seqlock);
    //读操作代码
    .....
} while(read_seqretry(&seqlock, seq_num));
```

八、 RCU (Read-Copy-Update)

RCU(**Read-Copy Update**), 是一种高级互斥机制。对于被 RCU 保护的共享数据结构, 读者不需要获得任何锁就可以访问它,但写者在访问它时首先拷贝一个副本,然后对副本进行修改,最后使用一个回调 (callback) 机制在适当的时机把指向原来数据的指针重新指向新的被修改的数据。

➤ RCU 锁的要素包括

❑ 读标志

- 如果有一个 Reader 企图更新 RCU 锁所保护的数据，Reader 只需设置一个标志，让外界知道有 Reader 在占据这把 RCU 锁，多个 Reader 可以共同持有一把 RCU 锁。

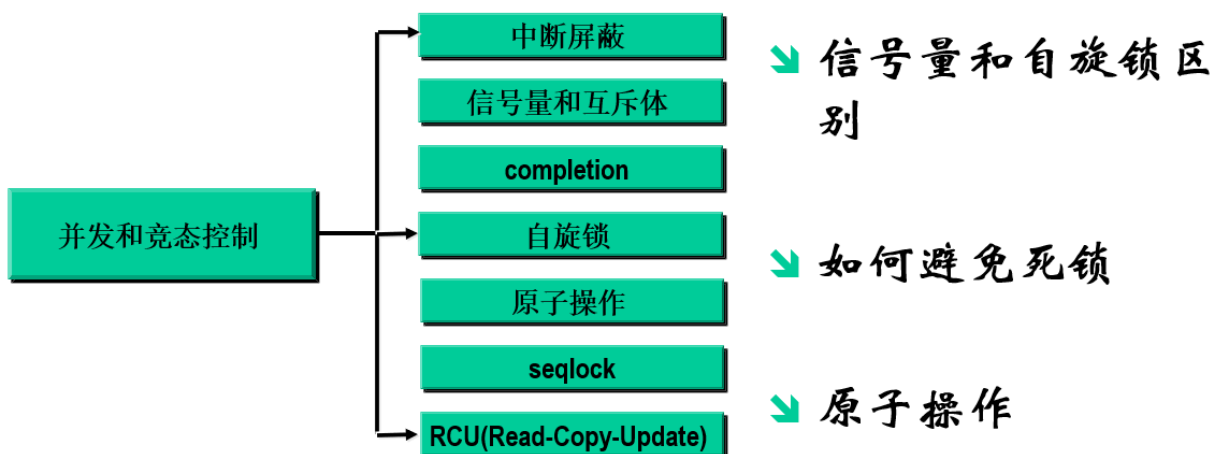
❑ 写时拷贝

- 如果有一个 Writer 企图更新 RCU 锁所保护的数据，那么它会首先查看该 RCU 锁的读标志，如果有该标志，说明有最少一个 Reader 持有了该 RCU 锁，它需要对原始数据 make a copy，写这个副本并将更新过的副本保存在某处，等待时机用该副本更新原始数据。

❑ 更新时机

- Writer 必须 waiting for all readers leaving，方可 Update 原始数据

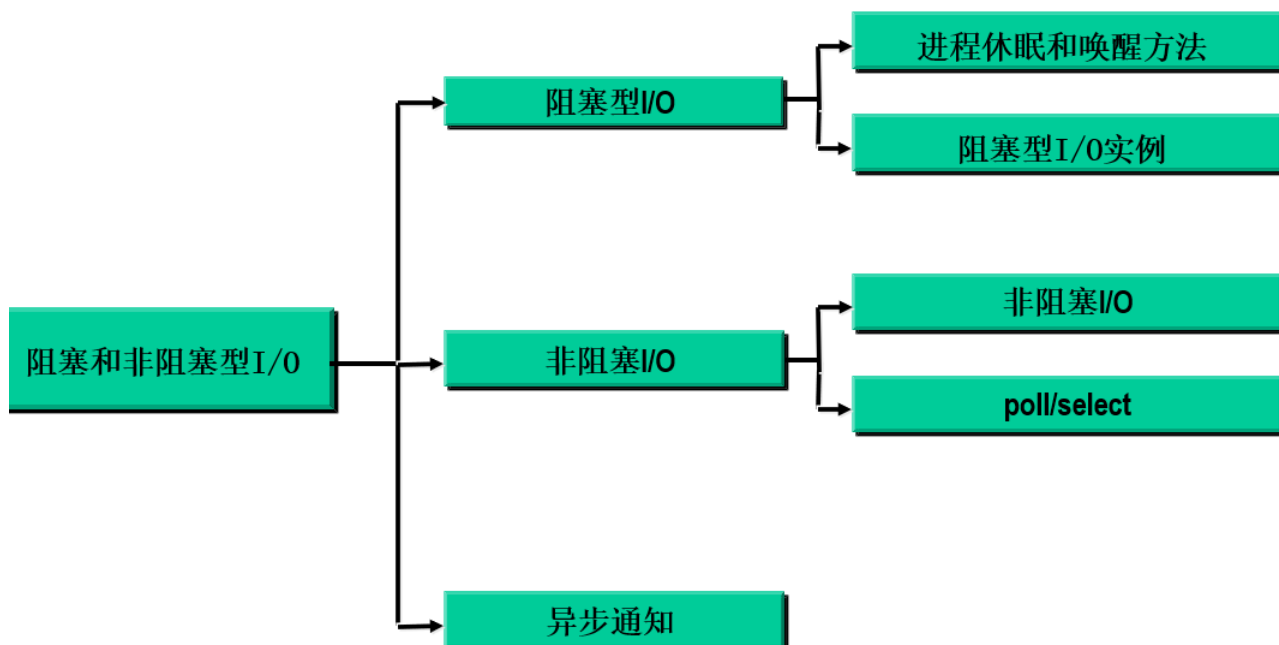
总结



第 11 讲 进程间通信机制分析

一、 阻塞/非阻塞型 I/O

➤ 阻塞型 I/O



当用户程序调用 `read` 函数时，驱动程序的 `read` 并没有准备好数据，怎么办？

1. 用户程序不会管理这些问题。

2. 驱动程序默认情况下，阻塞该进程。将其置入休眠状态，直到请求被满足，再唤醒该进程继续执行

➤ 休眠与唤醒

➤ 休眠的意义？

- ❑ 从调度器的运行队列 → 某个等待队列
- ❑ 等到某个事件发生，再从等待队列返回到运行队列。

➤ 如何将进程安全的进入休眠状态？

- ❑ 不能在原子上下文进行休眠：
- ❑ 休眠时对外界一无所知，进程必须重新检测等待条件
- ❑ 进程只有确保会 被其他进程唤醒，才能进入休眠

➤ 等待队列的作用

- ❑ 实现阻塞进程的唤醒
- ❑ 实现内核中的异步事件通知机制
 - 队列的数据结构
 - 系统的调度机制
- ❑ 同步系统资源的访问
 - Semaphore 可以用 wait queue 来实现

需要等待队列来管理和维护这些信息：

等待队列就是一个进程链表，其中包含了等待某个特定事件的所有进程

➤ 等待队列

➤ 等待队列通过“等待队列头”来管理

```
Struct wait_queue_head_t  
DECLARE_WAIT_QUEUE_HEAD(name);
```

```
wait_queue_head_t xxx_queue;  
init_waitqueue_head(&xxx_queue);
```

➤ 等待事件

- ❑ 在休眠的同时，也检查进程等待的条件

等待队列头，
通过“值传递”

➤ 休眠函数

```
wait_event(queue, condition);  
wait_event_interruptible(queue, condition);
```

休眠前后都要对该表达式求值；在条件为真之前，进程会保持休眠

```
wait_event_timeout(queue, condition, timeout);  
wait_event_interruptible_timeout(queue, condition, timeout);
```

➤ 唤醒函数

```
wake_up(wait_queue_head_t *queue);  
约定的做法是成对使用  
wake_up_interruptible(wait_queue_head_t *queue);
```

```
wake_up_nr(wait_queue_head_t *queue, int nr);  
wake_up_interruptible_nr(wait_queue_head_t *queue, int nr);
```

唤醒 `nr` 各独占等待进程，而不是一个，当 `nr=0` 时，唤醒所有的独占等待进程。

```
wake_up_all(wait_queue_head_t *queue);
wake_up_interruptible_all(wait_queue_head_t *queue);
```

➤ 等待队列示例

```
static DECLARE_WAIT_QUEUE_HEAD(wq);
static int flag = 0;
ssize_t sleepy_read(struct file *filp, char __user *buf,
size_t count, loff_t *pos)
{
    printk(KERN_DEBUG "process %i (%s) going to sleep\n",
        current->pid, current->comm);
    wait_event_interruptible(wq, flag != 0);
    flag = 0;
    printk(KERN_DEBUG "awoken %i (%s)\n", current->pid,
current->comm);
    return 0;
}

ssize_t sleepy_write(struct file *filp, const char __user *buf,
size_t count, loff_t *pos)
{
    printk(KERN_DEBUG "process %i (%s) awakening the
readers...\n", current->pid, current->comm);
    flag = 1;
    wake_up_interruptible(&wq);
    return count;
}
```

定义并初始化 wait_queue_head_t

在调用前后都要检查condition, 如果满足条件, 就不再休眠, 否则进入休眠状态

唤醒休眠在wq队列上的所有的进程。

➤ 上面实例中存在一个概率很小的竞态条件

- ❑ A,B 进程都等在 wq 的队列上
- ❑ C 进程调用 wait_event_interruptible
- ❑ A 进程被唤醒, 检查条件 flag !=0 成立
- ❑ 此时, 调度到 B 进程
- ❑ B 进程也检查到 flag != 0 成立,
- ❑ 这样, 一个事件唤醒了两个进程, 可能产生竞态

➤ 可以用原子操作防止这种情况

➤ 非阻塞型 I/O

- 调用进程显式的指明不想阻塞
- 设置 filp->f_flag |= O_NONBLOCK

```
int fd;
char buf;
fd = open("/dev/ttyS1", O_RDWR | O_NONBLOCK);
...
while(read(fd, &buf, 1) != 1); /*串口上无输入也返回,
                               所以要循环尝试读取串口*/
printf("%c\n", buf);
```

➤ Poll 和 select

- Poll、select、epoll 都允许进程决定是否可以对一个或多个打开的文件做非阻塞的读取或写入
- 这些调用也会阻塞进程, 直到给定的文件描述符集合中的任何一个可读取或写入
- 常常用于那些要使用多个输入或输出流而又不会阻塞于其中任何一个流的应用程序中

- 调用 poll 和 select 的目的是:
 - ❑ 确定接下来的操作是否是阻塞型的
- 使应用程序可以同时等待多个数据流
 - ❑ 从设备读取数据
 - read 可以返回比请求少的数据
 - 如果设置了 O_NONBLOCK, 在没有数据时, 立即返回。此时 poll 会报告设备不可读。
 - 如果到达文件尾, poll 会报告 POLLUP.
 - ❑ 向设备写数据
 - 写缓冲区有空间, poll 应该报告设备可写
 - 写缓冲区已满, poll 应该报告设备不可写
 - 永远不要让 write 调用在返回前等待数据的传输结束, 即使 O_NONBLOCK 标志被清除。

➤ select() 系统调用的原型如下:

```
int select ( int numfds, fd_set *readfds, fd_set *writefds,
            fd_set *exceptfds, struct timeval *timeout);
```

```
struct timeval
{
    int tv_sec; /*秒*/
    int tv_usec; /*微妙*/
};
```

➤ 文件描述符集合相关函数:

```
FD_ZERO(fd_set *set);
```

清除一个文件描述符集

```
FD_SET(int fd, fd_set *set);
```

将一个文件描述符加入文件描述符集中

```
FD_CLR(int fd, fd_set *set);
```

将一个文件描述符从文件描述符集中清除。

```
FD_ISSET(int fd, fd_set *set);
```

判断文件描述符是否被置位

```
int fd;
fd_set rfdset,wfdset;//读/写文件描述符集
/*以非阻塞方式打开/dev/xxx 设备文件*/
fd = open("/dev/xxx", O_RDWR | O_NONBLOCK);
FD_ZERO(&rfdset);
FD_ZERO(&wfdset);
FD_SET(fd, &rfdset);
FD_SET(fd, &wfdset);
select(fd + 1, &rfdset, &wfdset, NULL, NULL);
/*数据可获得*/
if(FD_ISSET(fd, &rfdset))
{
    //读数据}
if(FD_ISSET(fd, &wfdset))
{
    //写数据}
```

二、 驱动程序 Poll 机制分析

➤ Poll 系统调用

➤ poll()系统调用的原型如下:

```
int poll(struct pollfd fds[], nfds_t nfds, int timeout);
```

➤ 参数描述:

```
struct pollfd {
    int fd;           //表示文件描述符
    short events;     //表示请求检测的事件
    short revents;    //表示检测之后返回的事件
};
```

- `fds`: 存放需要被检测状态的文件描述符;
- `nfds`: 用于标记数组 `fds` 中的 `struct pollfd` 结构元素的总数量;
- `timeout`: `poll` 函数调用阻塞的时间, 单位是MS (毫秒)

➤ 驱动程序 Poll 机制分析

➤ 驱动程序poll()函数的原型如下:

```
unsigned int (*poll) (struct file *filp, struct poll_table *wait);
```

➤ 示例:

```
static unsigned int scull_p_poll(struct file *file, poll_table *wait)
{
    struct scull_pipe *dev = filp->private_data;
    unsigned int mask = 0;
    down(&dev->sem);
    poll_wait(file, &dev->inq, wait); // 不会立即休眠
    poll_wait(file, &dev->outq, wait); // 不会立即休眠
    if (dev->rp != dev->wp)
        mask |= POLLIN | POLLRDNORM; //标识设备可读取
    if (spacefree(dev))
        mask |= POLLOUT | POLLWRNORM; //标识设备可写入
    up(&dev->sem);
    return mask;
}
```

加入等待队列, 不会立即休眠

设置设备读写标志, mask值即为poll系统调用中revents值

➤ poll_wait()函数的原型如下:

```
static inline void poll_wait(struct file * filp, wait_queue_head_t *wait_address,
                             poll_table *p)
{
    if (p && wait_address)
        p->qproc(filp, wait_address, p);
}
```

➤ p->qproc就是__pollwait函数, 把当前进程挂入我们驱动程序里定义的一个队列里。

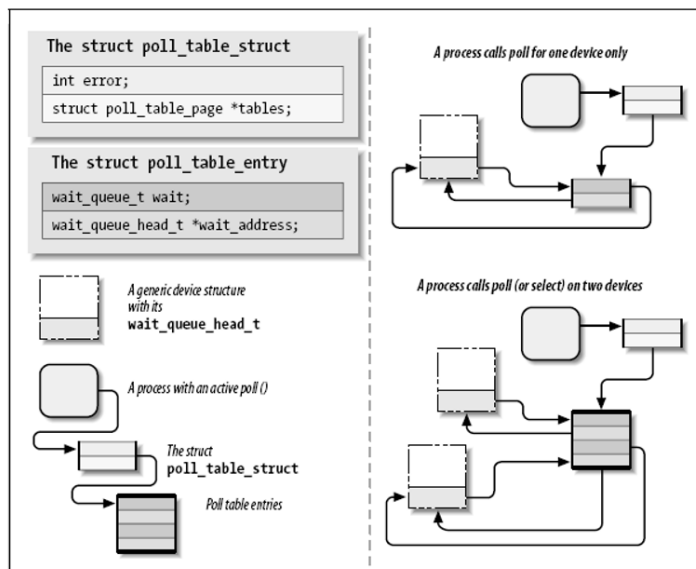


Figure 6-1. The data structures behind poll

➤ 系统调用 Poll 函数的执行分析

1. poll > sys_poll

```
asmlinkage long sys_poll(struct pollfd __user * ufds, unsigned int nfds, long timeout)
{
    /* Do a sanity check on nfds ... */
    poll_initwait(&table);

    /* Copy the events fields from user space */
    fdcount = do_poll(nfds, head, &table, timeout);

    /* OK, now copy the revents fields back to user space. */
    err = fdcount;
out_fds:
    /* Free temp memory. */
    poll_freewait(&table);
    return err;
}
```

poll_initwait初始化poll_wqueues table, 将其结构中的函数指针设置为__pollwait

遍历监听的文件描述符, 获取读写状态

清空等待队列

2. sys_poll > do_poll

```
static int do_poll(unsigned int nfds, struct poll_list *list, struct poll_wqueues *wait, long timeout)
{
    int count = 0;
    poll_table* pt = &wait->pt;

    if (!timeout) pt = NULL;
    for (;;) {
        struct poll_list *walk;
        set_current_state(TASK_INTERRUPTIBLE);
        walk = list;
        while(walk != NULL) {
            do_pollfd(walk->len, walk->entries, &pt, &count);
            walk = walk->next;
        }
        pt = NULL;
        if (count || !timeout || signal_pending(current)) break;
        count = wait->error;
        if (count) break;
        timeout = schedule_timeout(timeout);
    }
    set_current_state(TASK_RUNNING);
    return count;
}
```

遍历监听的fd, 获取event状态

监听到events或监听超时, 直接返回

未监听到events, 等待

3. do_poll > do_pollfd

```
static void do_pollfd(unsigned int num, struct pollfd *fdpage, poll_table **pwait, int *count)
{
    for (i = 0; i < num; i++) {
        int fd; unsigned int mask; struct pollfd *fdp;
        fdp = fdpage+i; fd = fdp->fd;
        if (fd >= 0) {
            struct file *file = fget(fd);
            mask = POLLNVAL;
            if (file != NULL) {
                mask = DEFAULT_POLLMASK;
                if (file->f_op && file->f_op->poll)
                    mask = file->f_op->poll(file, *pwait);
                mask &= fdp->events | POLLERR | POLLHUP;
                fput(file);
            }
            if (mask) {
                *pwait = NULL;
                (*count)++;
            }
            fdp->revents = mask;
        }
    }
}
```

调用驱动程序的poll函数, 获取设备状态

把获取到的设备状态与events比较, 确定是否监听到events

返回监听结果revents

三、 进程间通信机制分析

➤ 无名管道 Pipe

无名管道是 Linux 中进程间通信的一种方式。

- ✓ 它只能用于具有亲缘关系的进程之间的通信（也就是父子进程或者兄弟进程之间）。
- ✓ 它是一个半双工的通信模式，具有固定的读端和写端。
- ✓ 管道也可以看成是一种特殊的文件，对于它的读写也可以使用普通的 read()和 write()等函数。但是它不是普通的文件，并不属于其他任何文件系统，并且只存在于内核的内存空间中。

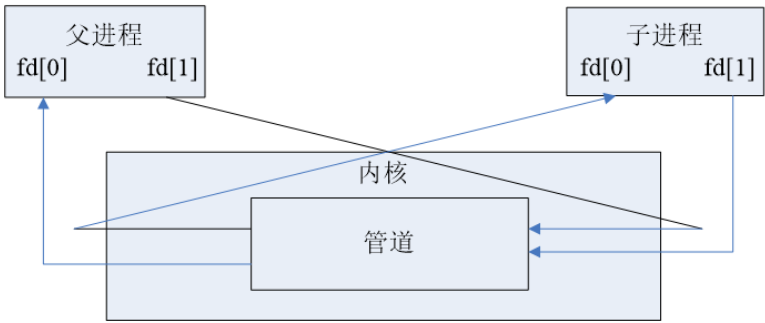
➤ 无名管道的创建与关闭

- ❑ 管道是基于文件描述符的通信方式，当一个管道建立时，它会创建两个文件描述符 `fds[0]`和 `fds[1]`，其中 `fds[0]`固定用于读管道，而 `fd[1]`固定用于写管道，这样就构成了一个半双工的通道。
- ❑ 创建管道可以通过调用 `pipe()`来实现。
- ❑ 管道关闭时只需使用普通的 `close()`函数逐个关闭各个文件描述符。

所需头文件	<code>#include <unistd.h></code>
函数原型	<code>int pipe(int fd[2])</code>
函数传入值	<code>fd[2]</code> : 管道的两个文件描述符，之后就可以直接操作这两个文件描述符
函数返回值	成功: 0 出错: -1

➤ 父子进程之间的无名管道通信

- ❑ 用 `pipe()`函数创建的管道两端处于一个进程中，由于管道是主要用于在不同进程间通信的，因此这在实际应用中并没有太大意义。实际上，通常先是创建一个管道，再通过 `fork()`函数创建一子进程，该子进程会继承父进程所创建的管道。



- ❑ 父子进程分别拥有自己的读写通道，为了实现父子进程之间的读写，只需把无关的读端或写端的文件描述符关闭即可。此时，父子进程之间就建立起了一条“子进程写入父进程读取”的通道。



- Pipe机制源码分析
 - ❑ pipe机制源码分析
[linux-source-3.2.0/fs/pipe.c](#)
 - ❑ pipe机制中poll机制源码分析
[linux-source-3.2.0/fs/pipe.c](#)

➤ 有名管道 FIFO

- 有名管道 FIFO 和一般的管道 Pipe 基本相同，但也有一些显著的不同：
 - ❑ 有名管道 FIFO 是在文件系统中作为一个特殊的设备文件而存在的。
 - ❑ 不同祖先的进程之间可以通过管道共享数据。
 - ❑ 当共享管道的进程执行完所有的 I / O 操作以后，有名管道将继续保存在文件系统中以便以后使用。
 - ❑ 管道只能由相关进程使用，它们共同的祖先进程创建了管道。但是，通过 FIFO，不相关的进程也能交换数据。

➤ 有名管道 FIFO 的创建：

int mkfifo(const char * pathname, mode_t mode);

- ❑ 返回：若成功则为 0，若出错则为 - 1
 - ❑ 一旦已经用 mkfifo 创建了一个 FIFO，就可用 open 打开它。确实，一般的文件 I / O 函数（close、read、write、unlink 等）都可用于 FIFO。
 - ❑ mkfifo 并不是一个系统调用，mkfifo()最终调用了 mknod()系统调用来执行实际的功能
 - ❑ FIFO 文件在磁盘上没有数据块，仅用来标识内核中的一条通道
- 由于普通文件的读写时不会出现阻塞问题，而在管道的读写中却有阻塞的可能，这里的非阻塞标志可以在 open()函数中设定为 O_NONBLOCK。
- 对于读进程
 - ❑ 若该管道是阻塞打开，且当前 FIFO 内没有数据，则对读进程而言将一直阻塞到有数据写入。
 - ❑ 若该管道是非阻塞打开，则不论 FIFO 内是否有数据，读进程都会立即执行读操作。即如果 FIFO 内没有数据，则读函数将立刻返回 0。
- 对于写进程
 - ❑ 若该管道是阻塞打开，则写操作将一直阻塞到数据可以被写入。
 - ❑ 若该管道是非阻塞打开而不能写入全部数据，则读操作进行部分写入或者调用失败。

➤ 消息队列

消息队列就是一些消息的列表。用户可以从消息队列中添加消息和读取消息等。从这点上看，消息队列具有一定的 FIFO 特性，但是它可以实现消息的随机查询，比 FIFO 具有更大的优势。同时，这些消息又是存在于内核中的，由“队列 ID”来标识。

- 消息队列的实现包括创建或打开消息队列、添加消息、读取消息和控制消息队列这四种操作。
 - ❑ 创建或打开消息队列使用的函数是 msgget()，这里创建的消息队列的数量会受到系统消息队列数量的限制；
 - ❑ 添加消息使用的函数是 msgsnd()函数，它把消息添加到已打开的消息队列末尾；
 - ❑ 读取消息使用的函数是 msgrcv()，它把消息从消息队列中取走，与 FIFO 不同的是，这里可以指定取走某一条消息；
 - ❑ 最后控制消息队列使用的函数是 msgctl()，它可以完成多项功能。

➤ msgget()

所需头文件	#include <sys/types.h>+ #include <sys/ipc.h>+ #include <sys/shm.h>+
函数原型	int msgget(key_t key, int msgflg)
函数传入值	key: 消息队列的键值，多个进程可以通过它访问同一个消息队列，其中有个特殊值 IPC_PRIVATE。它用于创建当前进程的私有消息队列。 msgflg: 权限标志位
函数返回值	成功: 消息队列 ID 出错: -1

➤ msgsnd()

所需头文件	#include <sys/types.h>+ #include <sys/ipc.h>+ #include <sys/shm.h>+	
函数原型	int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg)+	
函数传入值	msqid: 消息队列的队列 ID+	
	msgp: 指向消息结构的指针。该消息结构 msgbuf 通常为: + struct msgbuf+ {+ long mtype; /* 消息类型, 该结构必须从这个域开始 */+ char mtext[1]; /* 消息正文 */+ }+	
	msgsz: 消息正文的字节数 (不包括消息类型指针变量) +	
	msgflg: +	IPC_NOWAIT 若消息无法立即发送 (比如: 当前消息队列已满), 函数会立即返回。+ 0: msgsnd 调用阻塞直到发送成功为止。+
函数返回值	成功: 0+	
	出错: -1+	

➤ msgrcv()

所需头文件	#include <sys/types.h>+ #include <sys/ipc.h>+ #include <sys/shm.h>+	
函数原型	int msgrcv(int msqid, void *msgp, size_t msgsz, long int msgtyp, int msgflg)+	
函数传入值	msqid: 消息队列的队列 ID+	
	msgp: 消息缓冲区, 同于 msgsnd() 函数的 msgp+	
	msgsz: 消息正文的字节数 (不包括消息类型指针变量) +	
	msgtyp: +	0: 接收消息队列中第一个消息+
		大于 0: 接收消息队列中第一个类型为 msgtyp 的消息+
		小于 0: 接收消息队列中第一个类型值不小于 msgtyp 绝对值且类型值又最小的消息+
	msgflg: +	MSG_NOERROR: 若返回的消息比 msgsz 字节多, 则消息就会截短到 msgsz 字节, 且不通知消息发送进程+
		IPC_NOWAIT 若在消息队列中并没有相应类型的消息可以接收, 则函数立即返回+
		0: msgsnd() 调用阻塞直到接收一条相应类型的消息为止。
函数返回值	成功: 0+	
	出错: -1+	

msgctl()

所需头文件	#include <sys/types.h>+ #include <sys/ipc.h>+ #include <sys/shm.h>+	
函数原型	int msgctl (int msqid, int cmd, struct msqid_ds *buf)+	
函数传入值	msqid: 消息队列的队列 ID+	
	cmd: + 命令参数	IPC_STAT: 读取消息队列的数据结构 msqid_ds, 并将其存储在 buf 指定的地址中+
		IPC_SET: 设置消息队列的数据结构 msqid_ds 中的 ipc_perm 域 (IPC 操作权限描述结构) 值。这个值取自 buf 参数。
		IPC_RMID: 从系统内核中删除消息队列。
	buf: 描述消息队列的 msqid_ds 结构类型变量+	
函数返回值	成功: 0+	
	出错: -1+	

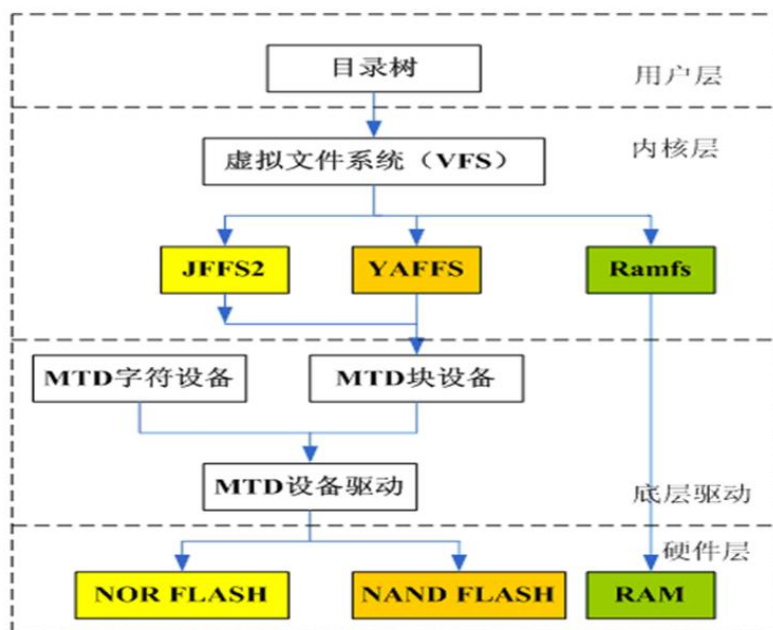
第 12 讲 嵌入式文件系统

一、 文件系统简介

Linux 支持多种文件系统，包括 ext2、ext3、vfat、ntfs、iso9660、jffs2、YAFFS、ramfs、romfs 和 nfs 等。

为了对各类文件系统进行统一管理，Linux 引入了虚拟文件系统 VFS(Virtual File System)，为各类文件系统提供一个统一的操作界面和应用编程接口。

Linux下的文件系统结构如下：

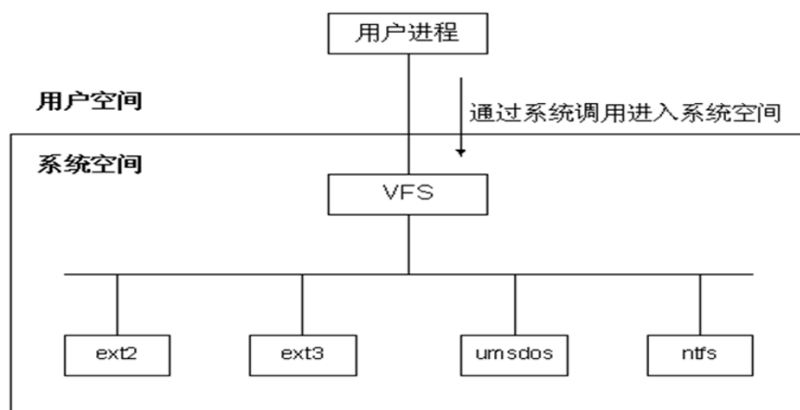


不同的文件系统类型有不同的特点，因而根据存储设备的硬件特性、系统需求等有不同的应用场合。

在嵌入式 Linux 应用中，主要的存储设备为 RAM(通常是 SDRAM)和 ROM(常采用 FLASH 存储器)，常用的基于存储设备的文件系统类型包括：jffs2, yaffs, cramfs, romfs, ramdisk, ramfs/tmpfs 等。

➤ 虚拟文件系统 (VFS)

- 虽然 Linux 内核是用 C 语言写的，但是其中借鉴了很多“面向对象”的思想
- **VFS 层**类似于面向对象理论中的“**抽象基类**”的概念，而下面的一个个具体文件系统就相当于这个抽象基类的“派生类”。
- 所有对文件的操作如 open, write, read 等在 VFS 中只是实现了一个类似于“纯虚函数”的接口，针对每种具体的文件系统，就会使用其派生类中被“改写”的“虚函数”。
- VFS 面向用户界面（系统调用）提供一个统一的编程接口。例如用户可以在不同的文件系统上创建文件，但所使用的函数或命令都是相同的。



二、 嵌入式文件系统

➤ Linux 根文件系统

- 概念：Linux 根文件系统包括 Linux 启动时所必须的目录和关键性的文件，例如 Linux 启动时都需要有 init 目录下的相关文件，在 Linux 挂载分区时 Linux 一定会找/etc/fstab 这个挂载文件等，根文件系统中还包括了许多的应用程序 bin 目录等，任何包括这些 Linux 系统启动所必须的文件都可以成为根文件系统。

目录名称	含义
/bin	存放二进制程序，如：ls, cp
/boot	存放系统启动的一些程序
/dev	存放设备文件
/etc	存放系统配置文件，如：group, profile
/home	用户根目录
/lib	存放库文件
/media	媒介的挂载点，如：闪存
/mnt	该目录用来为其他文件系统提供安装点
/opt	不随发行版本一起交付的程序
/proc	proc文件系统
/root	超级用户根目录
/sbin	存放超级用户运行的二进制文件
/src	存放一些服务的目录
/sys	sys文件系统的目录
/var	存放经常变化的文件，如临时文件等

Linux根文件系统——/dev

选择用devfs(设备文件系统)来取代固定的静态设备文件，则可免去寻找设备信息的麻烦。
内核源码树的Documentation/devices.txt文档就是静态设备主要和次要编号的正式信包来源。

基本的/dev条目

文件名	说明	类型	主编号	次编号	权限位
mem	物理内存存取	字符	1	1	600
null	null(黑洞)设备(译注1)	字符集	1	3	666
zero	以null byte(零值字节)为数据来源	字符	1	5	666
random	真随机数产生器	字符	1	8	644
try0	现行的虚拟控制台	字符	4	0	600
tty1	第一个虚拟控制台	字符	4	1	600
ttyS0	第一个UART串行端口	字符	4	64	600
tty	现行的TTY设备	字符	5	0	666
console	系统控制台	字符	5	1	600

➤ Linux 根文件系统引导过程

- 一般的系统先启动的是/sbin/init，而如同原始的主流 init，BusyBox 也可以处理系统的启动工作。
- BusyBox 的 init 尤其适合在嵌入式系统中使用。
 - ◆ /sbin/init 是/bin/busybox 的符号链接，所以 BusyBox 是目标板系统上执行的第一应用程序。当 BusyBox 知道调用它的目的是要执行 init，它会立即跳转到 init 进程。
- BusyBox 的 init 进程会依次进行以下工作：
 - ❑ 为 init 设置信号处理进程。
 - ❑ 初始化控制台。
 - ❑ 剖析 inittab 文件、/etc/inittab 文件。
 - ❑ 执行系统初始化的命令行。
 - ❑ BusyBox 在缺省情况下会使用/etc/init.d/rcS 命令行。
 - ❑ 执行所有会导致 init 暂停的 inittab 命令。
 - ❑ 执行所有仅执行一次的 inittab 命令。
 - ❑ 一旦完成以上工作，init 进程便会循环执行以下工作：
 - ❑ 执行所有终止时必须重新启动的 inittab 命令。
 - ❑ 执行所有终止时必须重新启动但启动前必须先询问过用户的 inittab 命令。

➤ 基于 FLASH 的文件系统

❑ Jffs2 Yaffs/Yaffs2 Cramfs Romfs

➤ 基于 RAM 的文件系统

❑ Ramdisk ramfs/tmpfs

➤ Jffs2

- JFFS 文件系统最早是由瑞典 Axis Communications 公司基于 Linux2.0 的内核为嵌入式系统开发的文件系统。**JFFS2 是 RedHat 公司基于 JFFS 开发的闪存文件系统**，最初是针对 RedHat 公司的嵌入式产品 eCos 开发的**嵌入式文件系统**，所以 JFFS2 也可以用在 Linux, uCLinux 中。

➤ Jffs2: 日志闪存文件系统版本 2

- ❑ 主要用于 **NOR 型闪存，基于 MTD 驱动层**，特点是：可读写的、支持数据压缩的、基于哈希表的日志型文件系统，并提供了崩溃/掉电安全保护，提供“写平衡”支持等。
- ❑ **缺点**主要是当文件系统已满或接近满时，因为垃圾收集的关系而使 jffs2 的运行速度大大放慢。

➤ Yaffs/Yaffs2

- yaffs/yaffs2 是专为嵌入式系统使用 **NAND 型闪存**而设计的一种日志型文件系统。与 jffs2 相比，它减少了一些功能(例如不支持数据压缩)，所以速度更快，挂载时间很短，对内存的占用较小。另外，它还是跨平台的文件系统，除了 Linux 和 eCos，还支持 WinCE, pSOS 和 ThreadX 等。
 - ❑ yaffs/yaffs2 自带 **NAND 芯片**的驱动，并且为嵌入式系统提供了**直接访问文件系统的 API**，用户可以不使用 Linux 中的 MTD 与 VFS，直接对文件系统操作。
 - ❑ **yaffs 与 yaffs2 的主要区别**在于，前者仅支持小页(512 Bytes) NAND 闪存，后者则可支持大页(2KB) NAND 闪存。同时，yaffs2 在内存空间占用、垃圾回收速度、读/写速度等方面均有大幅提升。

➤ Cramfs

- Cramfs 是 Linux 的创始人 Linus Torvalds 参与开发的一种只读的压缩文件系统。它也基于 MTD 驱动程序。
 - ❑ 在 cramfs 文件系统中，每一页(4KB)被单独压缩，可以随机页访问，其压缩比高达 2:1,为嵌入式系统节省大量的 **Flash 存储空间**，使系统可通过更低容量的 FLASH 存储相同的文件，从而降低系统成本。
 - ❑ Cramfs 文件系统以压缩方式存储，在运行时解压缩，所以**不支持**应用程序以 **XIP** 方式运行，所有的应用程序**要求被拷到 RAM 里去运行**。

➤ Romfs

- 传统型的 Romfs 文件系统是一种简单的、紧凑的、只读的文件系统，不支持动态擦写保存，按顺序存放数据，因而支持应用程序以 XIP(eXecute In Place, 片内运行)方式运行，在系统运行时，节省 RAM 空间。uCLinux 系统通常采用 Romfs 文件系统。

➤ Ramdisk

- Ramdisk 是**将一部分固定大小的内存当作分区**来使用。它并非一个实际的文件系统，而是一种**将实际的文件系统装入内存的机制**，并且**可以作为根文件系统**。将一些**经常被访问而又不会更改的文件**(如只读的根文件系统)通过 Ramdisk 放在内存中，可以明显地提高系统的性能。
- 在 Linux 的**启动阶段**，initrd 提供了一套机制，可以**将内核映像和根文件系统一起载入内存**。

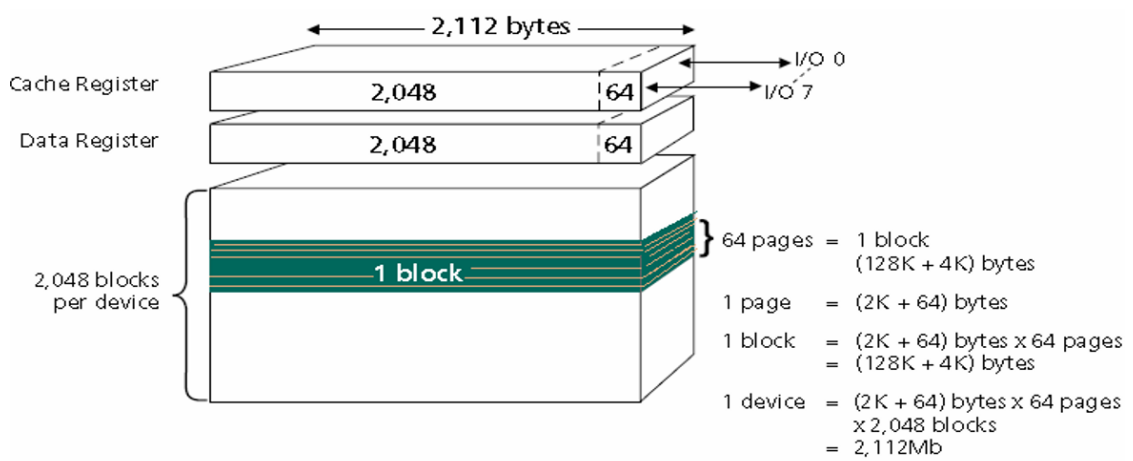
➤ ramfs/tmpfs

- Ramfs 是 Linus Torvalds 开发的一种基于内存的文件系统，工作于虚拟文件系统(VFS)层，不能格式化，可以创建多个，在创建时可以指定其最大能使用的内存大小。
- Ramfs/tmpfs 文件系统把所有的文件都放在 RAM 中，所以读/写操作发生在 RAM 中，可以用 ramfs/tmpfs 来存储一些临时性或经常要修改的数据，例如/tmp 和/var 目录，这样既避免了对 Flash 存储器的读写损耗，也提高了数据读写速度。
 - ❑ Ramfs/tmpfs 相对于传统的 Ramdisk 的不同之处主要在于：**不能格式化，文件系统大小可随所含文件内容大小变化。**
 - ❑ Tmpfs 的一个缺点是当**系统重新引导时会丢失所有数据。**

三、 Flash 存储器简介

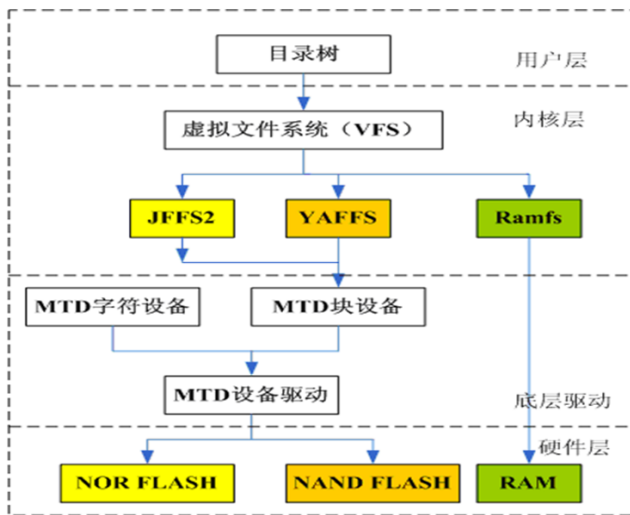
Flash 存储器是近几年来发展最快的存储设备，通常也称作闪存。Flash 属于 EEPROM（电可擦除可编程只读存储器），是一类存取速度很高的存储器。它既有 ROM 断电可保存数据的特点，又有易于擦写的特点。Flash 可以在断电的情况下长期保存信息，因此被广泛的应用在 PC 机的 BIOS 和嵌入式系统的存储设备。

- Flash Memory 是一种非易失性存储器，根据结构的不同可以将其分成 NOR Flash 和 NAND Flash 两种。
- Flash Memory 在物理结构上分成若干个区块，区块之间相互独立。NOR Flash 把整个存储区分成若干个扇区 (Sector)，而 NAND Flash 把整个存储区分成若干个块 (Block)，可以对以块或扇区为单位的内存单元进行擦写和再编程。
- 写操作只能将数据位从 1 写成 0，而不能从 0 写成 1，在对存储器进行写入之前必须先执行擦除操作，将预写入的数据位初始化为 1。
- 擦除操作的最小单位是一个区块
- NAND Flash 随机读取能力差，适合大数据的连续读取。
- NOR Flash 可以很容易地存取其内部的每一个字节
- NOR Flash 容量小，适用于代码存储
- NAND Flash 容量大，适用于数据存储
- 在 NOR Flash 上运行代码不需要任何的软件支持。在 NAND Flash 上进行同样操作时，通常需要驱动程序，也就是内存技术驱动程序 (MTD)。
- NAND Flash 和 NOR Flash 在进行写入和擦除操作时都需要 MTD。
- Flash Memory 器件存在位交换现象。位反转的问题多见于 NAND Flash，需使用 EDC/ECC（错误探测 / 错误纠正）算法，以确保可靠性。
- Flash Memory 在使用过程中可能导致某些区块的损坏，一旦损坏无法修复。NAND Flash 中的坏块是随机分布的，在出厂时就可能存在这样的坏块，需要对介质进行初始化扫描以发现坏块，并将坏块标记为不可用。

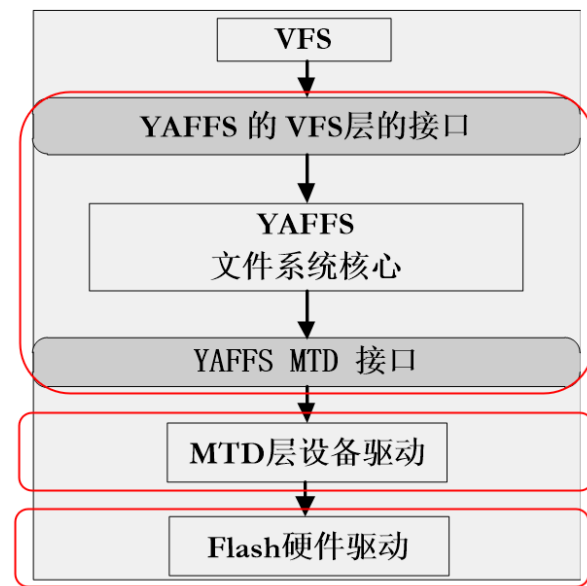


NAND Flash物理结构的相关数据示例

Linux下的文件系统结构如下:



YAFFS文件系统结构如下:

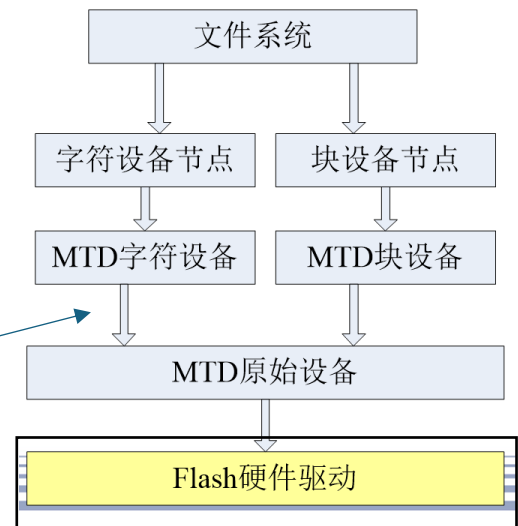


四、 NAND Flash 驱动

通过 `make menuconfig` 配置 NAND Flash

```
drivers/mtd/nand
drivers/mtd/nand/nand_base.c
drivers/mtd/nand/nand_bbt.c
drivers/mtd/nand/nand_ecc.c
drivers/mtd/nand/nand_ids.c
```

五、 MTD 驱动



MTD(Memory Technology Device , 内存技术设备)是用于访问 Memory 设备 (ROM、Flash) 的 Linux 的子系统。MTD 的主要目的是为了使新的 Memory 设备的驱动更加简单, 为此它在硬件和上层之间提供了一个抽象的接口。MTD 的所有源代码在 `/drivers/mtd` 子目录下。

➤ MTD 专业名词

- **OOB: out of band**, 某些内存技术支持 out-of-band 数据——如 NAND 每 512 字节的块有 16 个字节的 extra data, 用于纠错或元数据。
- **ECC: error correction**, 某些硬件不仅允许对 flash 的访问, 也有 ecc 功能, 所有 flash 器件都受位交

换现象的困扰。在某些情况下，一个比特位会发生反转或被报告反转了，就要采用 ECC 算法。

- **erasesize**: 一个 erase 命令可以擦除的最小块的大小
- **buswidth**: MTD 设备的接口总线宽度
- **devicetype**: 芯片类型，x8、x16 或者 x32

MTD 原始设备层

➤ MTD 原始设备层-mtd_info

- **mtd_info** 结构是 MTD 原始设备层的一个重要结构，该结构定义了大量的关于 MTD 的数据和操作，定义在 include/linux/mtd/mtd.h 头文件。mtd_info 结构成员主要由数据成员和操作函数两部分组成。

```
1 struct mtd_info
2 {
3
4     u_char type; // 内存技术的类型
5     u_int32_t flags; // 标志位
6     u_int32_t size; // mtd 设备的大小
7     u_int32_t erasesize; // 主要的擦除块大小 (同一个
8                             // mtd 设备可能有数种不同的 erasesize)
9     u_int32_t oobblock; // oob 块大小
10    u_int32_t oobsize; // oob 数据大小
11    u_int32_t ecctype; // ecc 类型
12    u_int32_t eccsize; // ecc 工作的范围
13
14    char *name;
15    int index; // 索引
16
17    // 不同的 erasesize 的区域
18    int numeraseregions; // 不同 erasesize 的区域的数目 (通常是 1)
19    struct mtd_erase_region_info *eraseregions;
20
21    u_int32_t bank_size;
22    struct module *module;
23    int(*erase)(struct mtd_info *mtd, struct erase_info *instr);
24    // 此 routine 用于将一个 erase_info 加入 erase queue
```

flags 字段包括MTD_ERASEABLE (可擦除)、MTD_WRITEB_WRITEABLE (可编程)、MTD_XIP (可片内执行)、MTD_OOB (NAND 带外数据)、MTD_ECC (支持自动ECC) 等

```
33 int(*read)(struct mtd_info *mtd, loff_t from, size_t len, size_t
*retlen,
34     u_char *buf); // 读 Flash
35 int(*write)(struct mtd_info *mtd, loff_t to, size_t len, size_t
*retlen,
36     const u_char *buf); // 写 Flash
37 int(*read_ecc)(struct mtd_info *mtd, loff_t from, size_t len,
size_t *retlen,
38     u_char *buf, u_char *eccbuf); // 带 ecc 的读
39 int(*write_ecc)(struct mtd_info *mtd, loff_t to, size_t len, size_t
*retlen,
40     const u_char *buf, u_char *eccbuf); // 带 ecc 的写
41 int(*read_oob)(struct mtd_info *mtd, loff_t from, size_t len,
size_t *retlen,
42     u_char *buf); // 读 out-of-band
43 int(*write_oob)(struct mtd_info *mtd, loff_t to, size_t len, size_t
*retlen,
44     const u_char *buf); // 写 out-of-band
45 /* iovec-based 读写函数，对于 NAND Flash 需要针对它定义 */
46 int(*readv)(struct mtd_info *mtd, struct iovec *vecs, unsigned long
count,
47     loff_t from, size_t *retlen);
48 int(*writev)(struct mtd_info *mtd, const struct iovec *vecs,
unsigned long
```



```

50
51  /* Sync */
52  void(*sync)(struct mtd_info *mtd);
53
54  /* 设备锁 */
55  int(*lock)(struct mtd_info *mtd, loff_t ofs, size_t len);
56  int(*unlock)(struct mtd_info *mtd, loff_t ofs, size_t len);
57  /* 能量管理函数*/
58  int(*suspend)(struct mtd_info *mtd);
59  void(*resume)(struct mtd_info *mtd);
60
61  void *priv; //私有数据
62 }

```

➤ MTD 原始设备层-mtd_table

- mtd_table (mtdcore.c) 则是所有 MTD 原始设备的列表
 - **struct mtd_info** *mtd_table[MAX_MTD_DEVICES];
 - ◆ 最多可以有 MAX_MTD_DEVICES (默认定义为 16) 个设备，每个 MTD 分区也算一个 MTD 设备。

```

int add_mtd_device(struct mtd_info *mtd);
int del_mtd_device (struct mtd_info *mtd);

```

add_mtd_device 添加 MTD 原始设备到 mtd_table 中

del_mtd_device 从 mtd_table 中删除 MTD 原始设备

➤ MTD 原始设备层-mtd_part

- mtd_part (mtdpart.c) 是用于表示 MTD 原始设备分区的结构，其中包含了 mtd_info，因为每一个分区都是被看成一个 MTD 原始设备加在 mtd_table 中的，mtd_part.mtd_info 中的大部分数据都从该分区的主分区 mtd_part->master 中获得。

```

1 struct mtd_part
2 {
3     struct mtd_info mtd; //分区的信息（大部分由其 master 决定）
4     struct mtd_info *master; //该分区的主分区
5     u_int32_t offset; //该分区的偏移地址
6     int index; //分区号
7     struct list_head list;
8 };

```

- add_mtd_partitions()会对每一个新建分区建立一个新的 mtd_part 结构体，将其加入 mtd_partitions 中，并调用 add_mtd_device()将此分区作为 MTD 设备加入 mtd_table。成功时返回 0，如果分配 mtd_part 时内存不足，则返回 -ENOMEM。
- del_mtd_partitions()的作用是针对 mtd_partitions 上的每一个分区，如果它的主分区是 master (参数 master 是被删除分区的主分区)，则将它从 mtd_partitions 和 mtd_table 中删除并释放掉，这个函数会调用 del_mtd_device()。

➤ 总结：MTD 原始设备层

□ 两种注册方式：

- 1) 直接注册整个 flash 设备 (MTD Device) 到 MTD。
 - ret = add_mtd_device(mtd);
- 2) 分 partition 添加到 mtd_table,并将每个 partition 当成一个 mtd 设备注册到 MTD。

- if (! (partitions && num_part > 0))
- ret = add_mtd_partitions(mtd, parts, num_part);

MTD (字符/块) 设备层

- 基于 MTD 原始设备, Linux 系统可以定义出 MTD 的块设备 (主设备号 31) 和字符设备 (设备号 90), 构成 MTD 设备层。
 - ❑ MTD 字符设备的定义在 mtdchar.c 中实现, 通过注册一系列 file_operation 函数 (lseek、open、close、read、write、ioctl) 可实现对 MTD 设备的读写和控制。
 - ❑ MTD 块设备则是定义了一个描述 MTD 块设备的结构 mtdblk_dev, 并声明了一个名为 mtdblks 的指针数组, 这数组中的每一个 mtdblk_dev 和 mtd_table 中的每一个 mtd_info 一一对应。

➤ MTD (字符/块) 设备层-字符设备

- mtdchar.c 实现了字符设备接口, 通过它, 用户可以**直接操作 Flash** 设备。
 - ❑ 通过 read()、write() 系统调用可以读写 Flash。
 - ❑ 通过一系列 IOCTL 命令可以获取 Flash 设备信息、擦除 Flash、读写 NAND 的 OOB、获取 OOB layout 及检查 NAND 坏块等。
 - MEMGETINFO、MEMERASE、MEMREADOOB、MEMWRITEOOB、MEMGETBADBLOCK、IOCRL
- 主要原理是将 Flash 的 erase block 中的数据在内存中建立映射, 然后对其进行修改, 最后擦除 Flash 上的 block, 将内存中的映射块写入 Flash 块。整个过程被称为 read/modify/erase/rewrite 周期。
- 但是, 这样做是不安全的, 当下列操作序列发生时, read/modify/erase/poweroff, 就会丢失这个 block 块的数据。
- 块设备模拟驱动按照 block 号和偏移量来定位文件, 因此在 Flash 上除了文件数据, 基本没有额外的控制数据。

➤ 在 linux 中, driver/mtd/mtdblock.c

```
struct mtdblk_dev {
    struct mtd_info *mtd; /* Locked */
    int count;
    struct semaphore cache_sem;
    unsigned char *cache_data;
    unsigned long cache_offset;
    unsigned int cache_size;
    erasesize)
    enum { STATE_EMPTY, STATE_CLEAN, STATE_DIRTY } cache_state;
} *mtdblks[MAX_MTD_DEVICES];
```

下层原始设备层的 MTD 设备结构

缓冲区 (与 MTD 设备块对齐)

缓冲区内数据在原始设备层 MTD 设备内的偏移

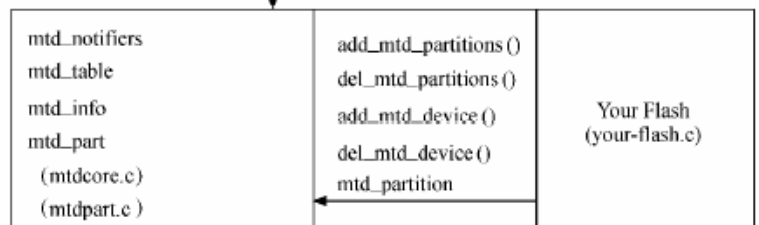
缓冲区大小 (通常被设置为 MTD 设备的

MTD 设备层



register_mtd_user()
get_mtd_device()
unregister_mtd_user()
put_mtd_device()
erase_info

原始设备层



➤ MTD 驱动总结

➤ Flash 硬件驱动

➤ 硬件驱动层负责在 init 时驱动 Flash 硬件

- ◆ Linux MTD 设备的 NOR Flash 芯片驱动遵循 CFI 接口标准，其驱动程序位于 drivers/mtd/chips 子目录下。
- ◆ NAND 型 Flash 的驱动程序则位于 /drivers/mtd/nand 子目录

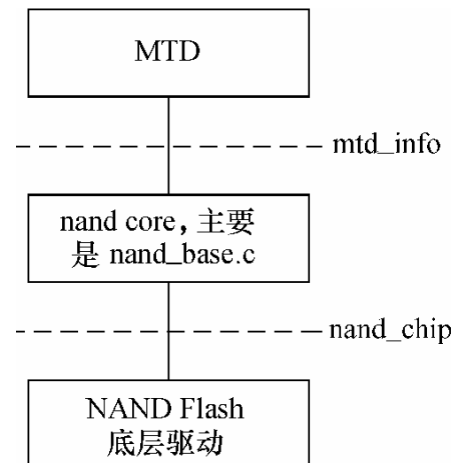
➤ Linux 内核在 MTD 的下层实现了通用的 NAND 驱动（主要通过 drivers/mtd/nand/nand_base.c 文件实现），因此芯片级的 NAND 驱动不再需要实现 mtd_info 中的 read()、write()、read_oob()、write_oob() 等成员函数，而主体转移到了 nand_chip 数据结构

➤ MTD 驱动源码分析

- 设备节点：/dev 下 MTD 字符设备节点（主设备号为 90）
- MTD 设备层：分为 MTD 字符设备层（mtdchar.c）和 MTD 块设备层（mtdblock.c），完成 flash 的基本操作。
- MTD 原始设备层：MTD 原始设备的通用代码（mtdcore.c），（mtdpart.c）其中 mtdcore.c 中定义了描述 mtd 设备的核心结构 mtdinfo。
- FLASH 硬件驱动层：
 - ◆ mtd/nand 目录，实现了对 flash 的读写擦校验等动作。

- 通过 make menuconfig 配置 MTD
- drivers/mtd

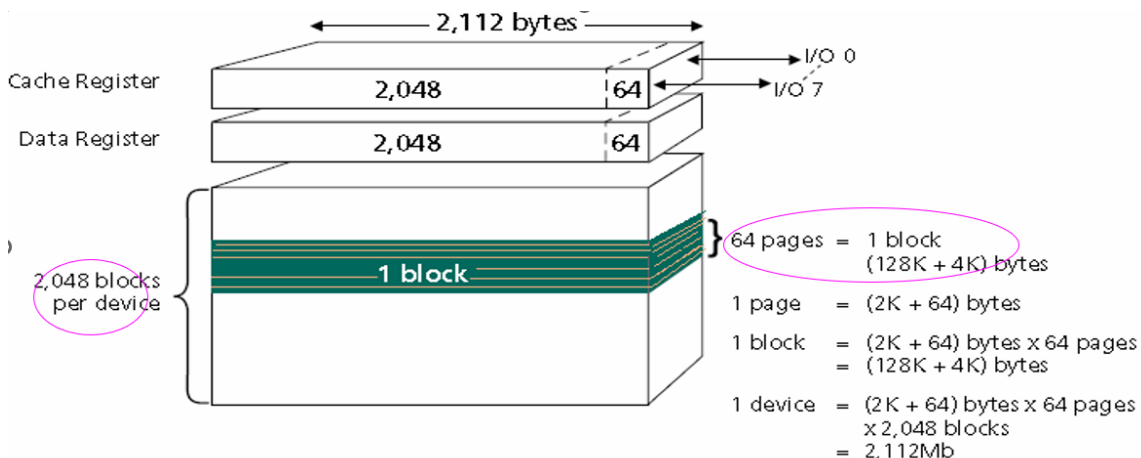
drivers/mtd/mtdchar.c
drivers/mtd/mtdblock.c
drivers/mtd/mtdcore.c
drivers/mtd/mtdpart.c



六、 YAFFS 文件系统

➤ YAFFS 的基本特征

YAFFS (Yet Another Flash File System) 专门为 **NAND Flash** 存储器开发的日志结构文件系统。

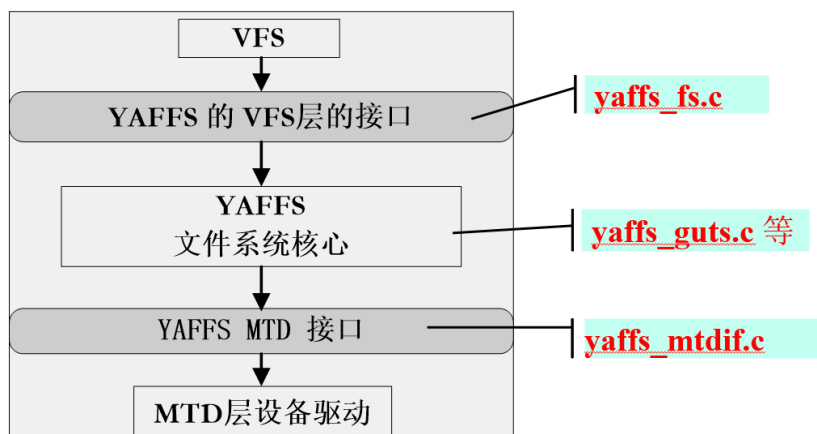


NAND Flash 特点及 YAFFS 的实现策略：

- NAND Flash 的读、写操作以页为单位，而擦除操作则是以块为单位进行；
 - YAFFS 文件是以固定大小的 chunk 进行存储的（chunk 的大小一般等于 NAND Flash 页的大小）

- ❑ NAND Flash 只能从 1 写为 0,不能从 0 写为 1;
- ❑ 每个擦除块的可擦除次数有限，一般为 100 万次;
- YAFFS 采用了日志结构文件系统的“异域更新”策略。每次按 Flash 页写入操作的时候，将新的元数据和用户数据作为日志记录追加到日志。
- YAFFS 对 NAND Flash 的废旧数据块回收的条件：1) 分配使用最早 2) 存在的无效数据页最多 3) 条件类似的情况下，进行随机选择
- 出厂时或使用过程都可能存在坏块，且随机分布;
 - ❑ 通过日志标识坏块或通过驱动层查询坏块，在 mount 时在内存中生成一个块状态表，记录块的状态
- 使用过程中可能会出现位反转(bit-flipping)的情况
 - ❑ YAFFS 提供了 ECC 校验算法或通过硬件提供

YAFFS 采用模块化设计的，分成以下几个部分：



- 通过 make menuconfig 配置 YAFFS
- fs/yaffs2
 - ❑ fs/yaffs2/yaffs_ecc.c
 - ❑ fs/yaffs2/yaffs_vfs.c
 - ❑ fs/yaffs2/yaffs_guts.c
 - ❑ fs/yaffs2/yaffs_mtdif.c

第 13 讲 编译、运行及版本管理

一、编译过程分析

➤ 编译预处理

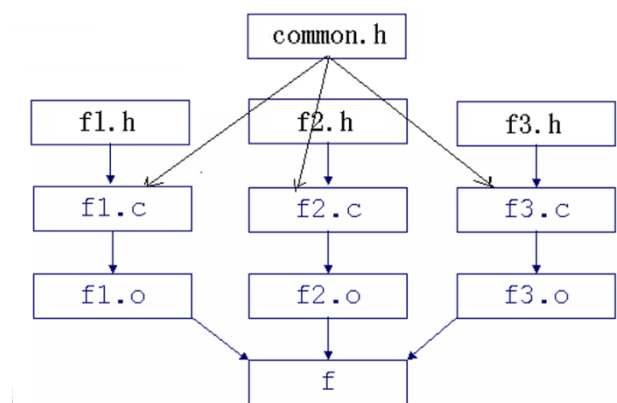
在预处理阶段，输入的是 C 语言的源文件，主要处理源文件中的 #ifdef、#include 和 #define 等命令，生成中间文件 *.i。

命令示例：gcc -E test.c -o test.i

➤ 编译

在编译阶段，输入的是中间文件 *.i，编译后生成汇编语言文件 *.s。

命令示例：gcc -S test.i -o test.s



➤ 汇编

在汇编阶段，将输入的汇编文件*.s
转换成机器语言*.o。

命令示例：gcc -c test.s -o test.o

➤ 链接

在链接阶段，将*.o 文件及其它库文件
汇集成一个可执行的二进制文件。

命令示例：gcc test.o -o test

➤ 编译命令

➤ 编译

- ❑ 编译时，gcc 首先调用预处理程序(cpp)对输入的源程序进行处理，然后调用 cc1 将预处理后的程序编译成汇编代码

➤ 汇编

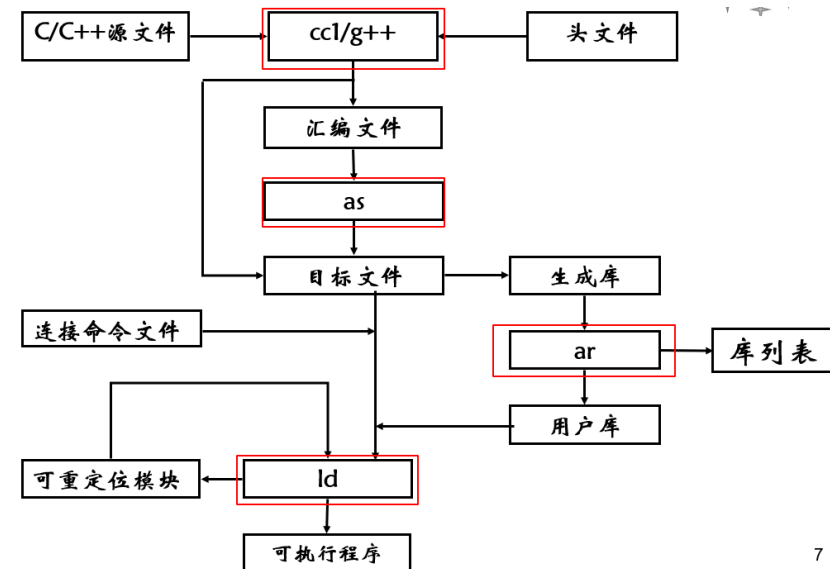
- ❑ as 将汇编语言程序转换为 ELF (Executable and Linking Format, 执行时链接文件格式)格式的可重定位目标代码，这些目标代码同其它目标模块或函数库易于定位和链接。
- ❑ as 产生一个交叉参考表和一个标准的符号表，产生的代码和数据能够放在多个区 (Section)中。

➤ 库管理

- ❑ ar 将多个可重定位的目标模块归档为一个函数库文件。
- ❑ 采用函数库文件，应用程序能够从该文件中自动装载要参考的函数模块，同时将应用程序中频繁调用的函数放入函数库文件中，易于应用程序的开发管理。

➤ 链接

- ❑ ld 根据链接定位文件 Linkcmds 中的代码区、数据区、BSS 区和栈区等定位信息，将可重定位的目标模块链接成一个单一的、绝对定位的目标程序。



二、 目标文件分析

➤ 目标文件是什么样的？

- 目标文件中的内容至少有编译后的机器指令代码、数据。还包括了链接时所需要的一些信息，比如符号表、调试信息、字符串等。
- 符号修饰标准、变量内层布局、函数调用方式等这些跟可执行代码二进制兼容性相关的内容称为 ABI (Application Binary Interface)。常见的 ABI 格式：

一般目标文件将这些信息按不同的属性，
以“节” (Section) 的形式存储，
有时候也叫“段” (Segment)

➤ 目标文件格式

- A.out
 - ◆ 不容易支持动态链接
 - ◆ 不支持 C++ 语言
- COFF
 - ◆ 非扩展版本不支持 C++ 和动态链接

ELF文件的两个视角

- ELF
 - ◆ 支持 C++
 - ◆ 支持动态链接
 - ◆ 允许交叉编译和交叉链接

ELF文件的三种类型

➤ 可重定位

- 编译器和汇编器创建
- 运行前需要被链接器处理

➤ 可执行

- 完成了所有重定位工作和符号解析
- 除了运行时解析的共享库符号

➤ 共享库

- 链接器需要的符号信息
- 运行时可以直接执行的代码

➤ 可重定位目标文件

- 一个典型的 ELF 可重定位文件
 - ◆ ELF 文件头部
 - ◆ 一系列的节
 - ◆ 节头部表
 - ◆ 段头部 (可选的)

Linking View	Execution View
ELF header	ELF header
Program header table <i>optional</i>	Program header table
Section 1	Segment 1
...	
Section <i>n</i>	Segment 2
...	...
Section header table	Section header table <i>optional</i>

ELF头部
.text
.rodata
.data
.bss
.sym
.rel.txt
.rel.data
.line
.debug
.strtab
节头部表

```
#define EI_NIDENT 16

typedef struct {
    unsigned char  e_ident[EI_NIDENT];
    Elf32_Half    e_type;
    Elf32_Half    e_machine;
    Elf32_Word    e_version;
    Elf32_Addr    e_entry;
    Elf32_Off     e_phoff;
    Elf32_Off     e_shoff;
    Elf32_Word    e_flags;
    Elf32_Half    e_ehsize;
    Elf32_Half    e_phentsize;
    Elf32_Half    e_phnum;
    Elf32_Half    e_shentsize;
    Elf32_Half    e_shnum;
    Elf32_Half    e_shstrndx;
} Elf32_Ehdr;
```

➤ ELF 文件头

- 第一部分：16 个字节的 e_ident, 描述了生成该文件的系统的字的大小和字节顺序，独立于 ELF 文件中的其它信息。

- 幻数：4 个字节，0x7f、'E'、'L'、'F'
- class：1 个字节，1=32 位，2=64 位
- data：1 个字节，1=小端，2=大端
- version：1 个字节，1=当前版本
- pad：剩余字节

- 第二部分：帮助链接器语法分析、解释目标文件的信息

- e_type：1=重定位、2=可执行、3=共享库
- e_machine：0=No machine、1=AT&T WE 32100、2=sparc、3=Intel 80386
- e_version：0=Invalid version、1=Current version
- e_ehsize：ELF 头部的大小 (以字节为单位)
- e_flags：体系结构相关标志，总是 0

```
#define EI_NIDENT 16

typedef struct {
    unsigned char  e_ident[EI_NIDENT];
    Elf32_Half    e_type;
    Elf32_Half    e_machine;
    Elf32_Word    e_version;
    Elf32_Addr    e_entry;
    Elf32_Off     e_phoff;
    Elf32_Off     e_shoff;
    Elf32_Word    e_flags;
    Elf32_Half    e_ehsize;
    Elf32_Half    e_phentsize;
    Elf32_Half    e_phnum;
    Elf32_Half    e_shentsize;
    Elf32_Half    e_shnum;
    Elf32_Half    e_shstrndx;
} Elf32_Ehdr;
```

可执行文件相关的表项

- ☐ e_entry: 程序的入口地址, 如无入口地址则该项为 0
- ☐ e_phoff: 程序头部表偏移量, 若无则为 0
- ☐ e_phentsize: 程序头部表中单个 entry 的大小
- ☐ e_phnum: 程序头部表中的 entry 个数

```
#define EI_NIDENT      16

typedef struct {
    unsigned char    e_ident[EI_NIDENT];
    Elf32_Half       e_type;
    Elf32_Half       e_machine;
    Elf32_Word       e_version;
    Elf32_Addr       e_entry;
    Elf32_Off        e_phoff;
    Elf32_Off        e_shoff;
    Elf32_Word       e_flags;
    Elf32_Half       e_ehsize;
    Elf32_Half       e_phentsize;
    Elf32_Half       e_phnum;
    Elf32_Half       e_shentsize;
    Elf32_Half       e_shnum;
    Elf32_Half       e_shstrndx;
} Elf32_Ehdr;
```

```
typedef struct {
    unsigned char    e_ident[EI_NIDENT];
    Elf32_Half       e_type;
    Elf32_Half       e_machine;
    Elf32_Word       e_version;
    Elf32_Addr       e_entry;
    Elf32_Off        e_phoff;
    Elf32_Off        e_shoff;
    Elf32_Word       e_flags;
    Elf32_Half       e_ehsize;
    Elf32_Half       e_phentsize;
    Elf32_Half       e_phnum;
    Elf32_Half       e_shentsize;
    Elf32_Half       e_shnum;
    Elf32_Half       e_shstrndx;
} Elf32_Ehdr;
```

重定位文件相关的表项

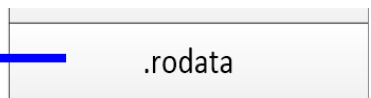
- ☐ e_shoff: 节头部表偏移量, 若无则为 0
- ☐ e_shentsize: 节头部表中单个 entry 的大小
- ☐ e_shnum: 节头部表中的 entry 个数
- ☐ e_shstrndx: 节名字串表所在的节 index

ELF 文件中的节

- .text: 已编译程序的机器代码, 具有 ALLOC 和 EXECINSTR 属性的 PROGBITS 类型区段, 相当于 a.out 的文本段
 - PROGBITS 类型: 程序内容, 包括代码, 数据和调试器信息
 - ALLOC 属性: 程序加载时该区段需要占用内存空间
 - EXECINSTR 属性: 该区段包含可执行的机器代码



- .rodata: 只读数据, 具有 ALLOC 属性和 PROGBITS 类型区段。由于是只读数据, 因此没有 WRITE 属性。



- .data: 具有 ALLOC 和 WRITE 属性的 PROGBITS 类型区段。对应于 a.out 的数据段



- .bss: 具有 ALLOC 和 WRITE 属性的 NOBITS 类型区段。BSS 区段在文件中没有分配空间, 因此是 NOBITS 类型, 但由于会在运行时分配空间, 所以具有 ALLOC 属性。



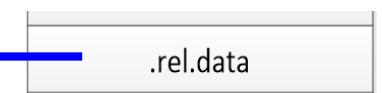
- .sym: 符号表。存放程序中定义和引用的 函数和全局变量 的信息。



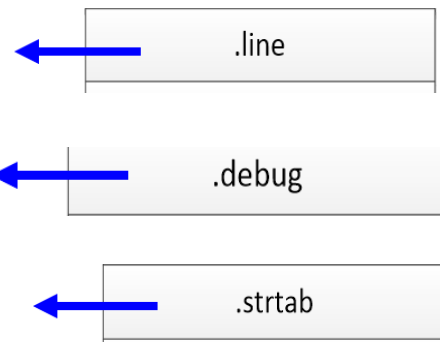
- .rel.txt: 一个 .text 节中位置的列表, 当链接器把这个目标文件和其它文件结合时, 需要修改这些位置。



- .rel.data: 一个 .data 节中位置的列表, 当链接器把这个目标文件和其它文件结合时, 需要修改这些位置。



- .line：源码到目标代码行号映射，
只有是使用-g 选项调用进行编译时才会得到这张表
- .debug：一个调试用的符号表，
包含了定义的局部变量和类型、全局变量的定义和引用
- .strtab：一个字符串表，其内容包括
.symtab 和.debug 节中的符号表，以及节头部中的节名字。



```
typedef struct {
    Elf32_Word    sh_name;
    Elf32_Word    sh_type;
    Elf32_Word    sh_flags;
    Elf32_Addr    sh_addr;
    Elf32_Off     sh_offset;
    Elf32_Word    sh_size;
    Elf32_Word    sh_link;
    Elf32_Word    sh_info;
    Elf32_Word    sh_addralign;
    Elf32_Word    sh_entsize;
} Elf32_Shdr;
```



➤ 节头表表项

```
typedef struct {
    Elf32_Word    sh_name;
    Elf32_Word    sh_type;
    Elf32_Word    sh_flags;
    Elf32_Addr    sh_addr;
    Elf32_Off     sh_offset;
    Elf32_Word    sh_size;
    Elf32_Word    sh_link;
    Elf32_Word    sh_info;
    Elf32_Word    sh_addralign;
    Elf32_Word    sh_entsize;
} Elf32_Shdr;
```

- sh_name：节名在节字符串表中的索引
- sh_type：节的属性
- sh_flags：节的标志
- sh_addr：若该节可加载，
则为该节第一个字节的地址，
否则为0
- sh_offset：节起始点在文件
中的起始位置
- sh_size：节大小（字节为单
位）

```
typedef struct {
    Elf32_Word    sh_name;
    Elf32_Word    sh_type;
    Elf32_Word    sh_flags;
    Elf32_Addr    sh_addr;
    Elf32_Off     sh_offset;
    Elf32_Word    sh_size;
    Elf32_Word    sh_link;
    Elf32_Word    sh_info;
    Elf32_Word    sh_addralign;
    Elf32_Word    sh_entsize;
} Elf32_Shdr;
```

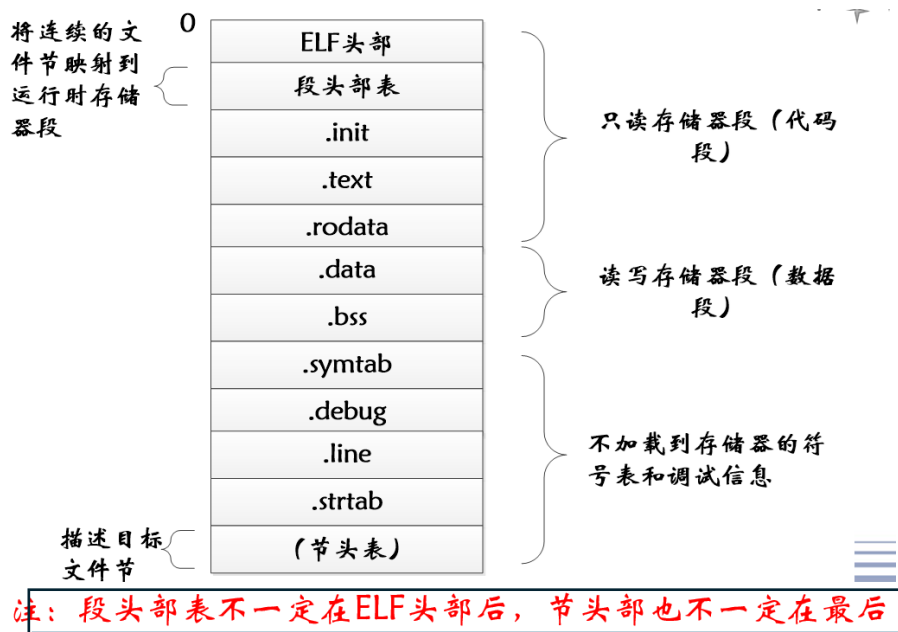
- sh_link：相关信息对应的节
号，若没有则为0
- sh_info：节的其它信息
- sh_align：节的对齐粒度
- sh_entsize：若节为一个表
时表项的大小

➤ ELF段头部表项

```
int type;           //类型：可加载代码或数据
int offset;         //段在文件中的偏移量
int virtaddr;       //映射段的虚拟地址
int physaddr;       //物理地址，未使用
int filesize;       //文件中的段大小
int memsize;        //内存中的段大小（如果包含BSS的话会更大些）
int flags;          //读、写、执行标志位
int align;          //对齐要求，根据硬件页尺大小不同而又变动
```

➤ ELF 可执行文件





➤ ELF 文件的重定位

➤ r_offset 的含义：

- ❑ 可重定位文件：从节开始到受重定位影响的区域首部的偏移量
- ❑ 可执行文件或共享库：受重定位影响的单元的虚拟地址

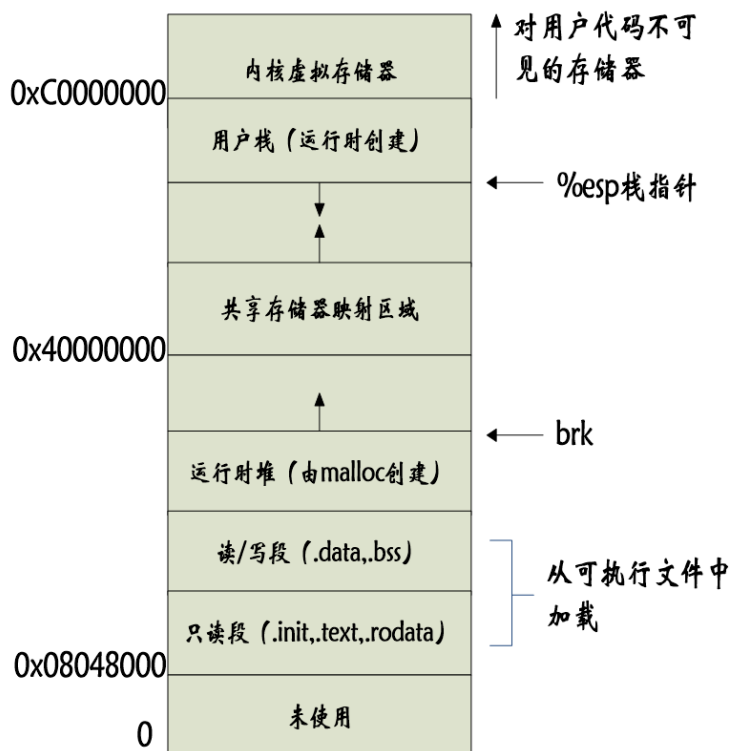
➤ r_info 的含义：

- ❑ 24 位的 symbol 域：重定位项在符号表中的索引
- ❑ 8 位的 type 域：指示如何进行重定位

```
typedef struct {
    Elf32_Addr    r_offset;
    Elf32_word    r_info;
} Elf32_Rel;
```

当加载器运行时，
创建了右图所示的存储器映像。

在可执行文件中
段头表的指导下，将
可执行文件中的相关
内容拷贝到 **代码和数据段**。



三、 运行过程分析

➤ 程序从 main 开始吗？

在 main 之前，main 函数的两个参数 (argc 和 argv) 已被正确传了进来。堆和 I/O 的初始化已经完成。

```
#include<stdio.h>#include<stdlib.h>
int main(int argc, char * argv[])
```

```
{int *p = (int *)malloc(sizeof(int));
scanf("%d",p);
printf("%d",*p);
free(p);
return 0;}
```

```
#include<stdio.h>
#include<string>
using namespace std;
string v; // v的构造函数已经在main之前被调用了
int foo(){
    int n; scanf("%d",&n); return n;
}
const int a = foo(); // a的初始化和foo函数调用也在main前进行
int main(){
    printf("%d\n",a);
    return 0;
}
```

➤ 入口函数和程序初始化

- 操作系统在创建进程后，把控制权交到了程序的入口，这个入口往往是运行库中的某个入口函数。
- 入口函数对运行库和程序运行环境进行初始化，包括堆、I/O、线程、全局变量构造，等等。
- 入口函数在完成初始化之后，调用 main 函数，正式开始执行程序主体部分。
- main 函数执行完毕之后，返回到入口函数，入口函数进行清理工作，包括全局变量析构、堆销毁、关闭 I/O 等，然后进行系统调用结束进程。

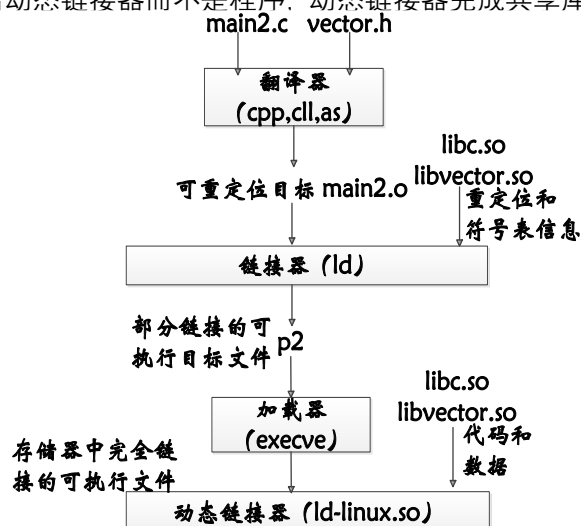
➤ C 程序入口函数

- C 程序的入口点，是符号_start 的地址。在_start 地址处的启动代码 (startup_code) 是在目标文件 crt1.o 中定义的，对所有的 C 程序都一样。
- 下图展示了启动代码的调用序列（但省略了将函数参数压入栈的代码），atexit 注册程序终止时应调用的程序，exit 函数调用 atexit 注册的代码，而后从_exit 返回 os。

```
1 0x080480c0 <_start>: /* entry point in .text */
2 call _libc_init_first /* startup code in .text */
3 call _init /* startup code in .init */
4 call atexit /* startup code in .text */
5 call main /* application main routine
6 call _exit /* return control to OS */
7 /* control never reaches here */
```

➤ 动态链接共享库

- 当目标文件和共享库一起编译时就能生成使用共享库的可执行程序。
- 对于使用共享库的可执行程序，文件中会有一.interp 节，该节包含了动态链接器的路径。
- 加载器将控制传递给动态链接器而不是程序，动态链接器完成共享库的重定位。最后，将控制传递给应用程序。

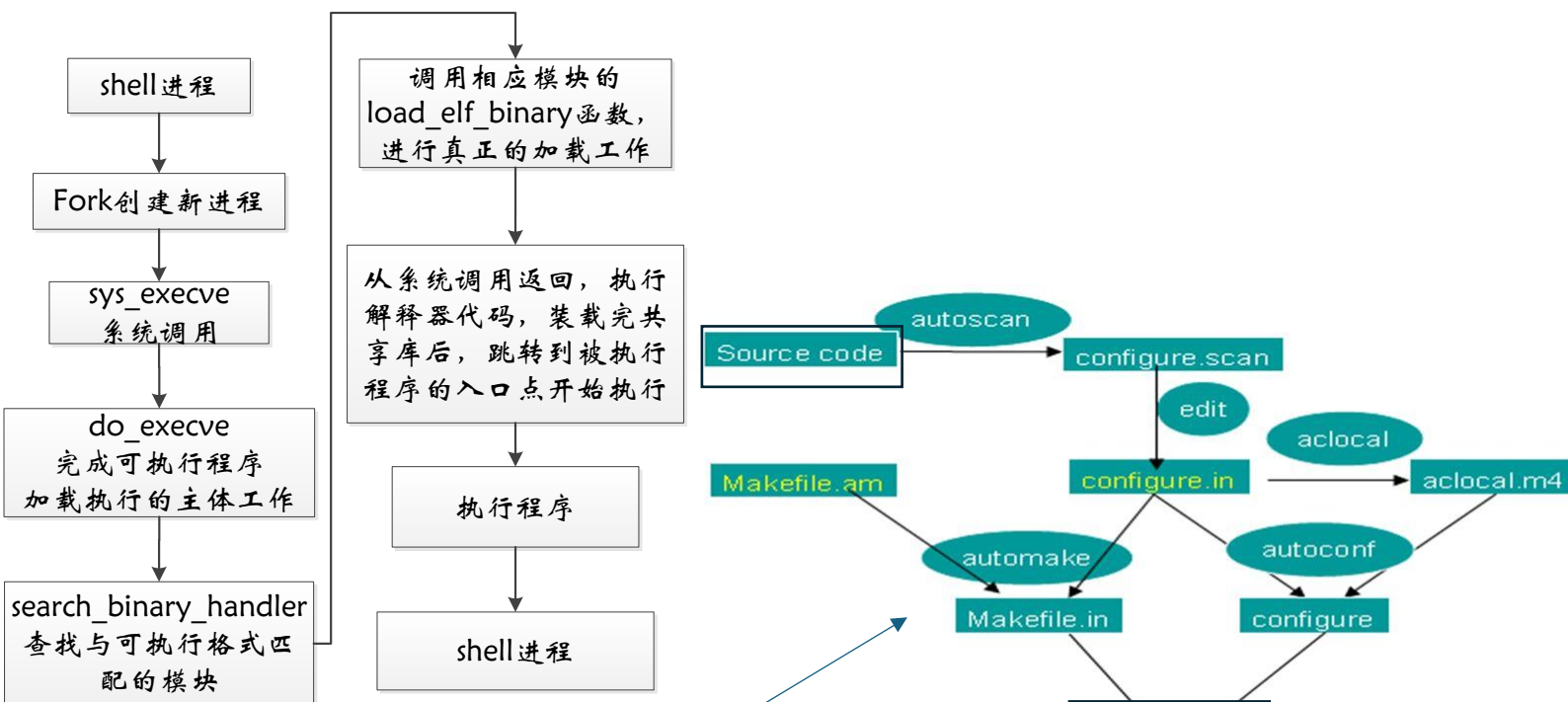


➤ 动态链接器(dynamic linker)

- 动态链接器执行以下步骤完成从程序到进程映像：
 - ◆ 分析可执行文件中的动态信息 section，决定需要哪些共享库。
 - ◆ 定位和映射(map)那些共享库，并且分析它们动态信息 section 决定是否需要附加的共享库。
 - ◆ 为可执行程序 and 那些需要的共享库执行重定位。
 - ◆ 调用共享库中提供的初始化函数并且安排共享库提供的清除(cleanup)函数在共享库卸载出进程空间的时候运行。
 - ◆ 将控制权传给程序

➤ ELF 文件加载与执行

- 首先，shell 进程通过 **fork** 系统调用，创建出一个新进程用来执行这个程序。
- 接着，新进程通过 **execve** 系统调用来**加载可执行文件**
 - ◆ execve 进行一系列操作后结束，此时，新进程的上下文及内存已经初始化完毕，当 CPU 从该系统调用返回到用户空间时，就开始运行可执行文件中的代码。
- 最终，用户程序执行完毕，shell 进程被切换上台。



四、 自动编译

➤ Autotools 简介

- Autoscan (autoscan)
 - ❑ 扫描源代码以搜寻普通的可移植性问题，比如检查编译器，库，头文件等，生成文件 configure.scan，它是 configure.in 的一个雏形
- Aclocal
 - ❑ 根据已经安装的宏，用户定义宏和 acinclude.m4 文件中的宏将 configure.in 文件所需要的宏集中定义到文件 aclocal.m4 中。
 - ❑ aclocal 是一个 perl 脚本程序，它的定义是：“aclocal - create aclocal.m4 by scanning configure.in”
- Automake
 - ❑ automake 根 Makefile.am 中定义的结构建立 Makefile.in

- Autoconf
 - ❑ 将 configure.in 中的宏展开，生成 configure 脚本。这个过程可能要用到 aclocal.m4 中定义的宏。

➤ **Configure.in 文件**

- 文件内容是一系列 GNU m4 的宏，这些宏经过 autoconf 工具处理后变成检查系统特性的 shell script。
- 文件里的宏的顺序没有特别的规定：但是一定以宏 AC_INIT 开始，以宏 AU_OUTPUT 结束
- 文件内容一般布局如下：
 - AC_INIT 测试程序 测试函数库 测试头文件 测试类型定义 测试结构
 - 测试编译器特性 测试库函数 测试系统调用 AC_OUTPUT

➤ **Makefile.am 文件**

Makefile.am 文件中几个预定的选项的含义如下所示。

文件类型	书写格式	变量	含义				
可执行文件	bin_PROGRAMS = foo foo_SOURCES =xxxxx.c foo_LDADD = foo_LDFLAGS = foo_DEPENDENCIES =	INCLUDES	比如链接时所需要的头文件				
		LDADD	比如链接时所需要的库文件				
		LDFLAGS	比如链接时所需要的库文件选项标志				
		EXTRA_DIST	源程序和一些默认的文件将自动打入.tar.gz包，其它文件若要进入.tar.gz包可以用这种办法，比如配置文件，数据文件等等。				
静态库	lib_LIBRARIES = libfoo.a foo_a_SOURCES = foo_a_LDADD = foo_a_LIBADD = foo_a_LDFLAGS =	SUBDIRS	在处理本目录之前要递归处理哪些子目录				
		<table><tr><th>路径变量</th><th>含义</th></tr><tr><td>\$(top_srcdir)</td><td>工程最顶层目录，用于引用源程序</td></tr><tr><td>\$(top_builddir)</td><td>定义了生成目标文件上最上层目录，用于引用.o等编译出来的目标文件。</td></tr></table>		路径变量	含义	\$(top_srcdir)	工程最顶层目录，用于引用源程序
路径变量	含义						
\$(top_srcdir)	工程最顶层目录，用于引用源程序						
\$(top_builddir)	定义了生成目标文件上最上层目录，用于引用.o等编译出来的目标文件。						
头文件	include_HEADERS = foo.h						
数据文件	data_DATA = data1 data2						

➤ **Makefile**

- 执行 configure 脚本文件所产生的 Makefile 文件有几个预定的选项可供使用。
 - ❑ **make all**：产生设定的目标，即生成所有的可执行文件。使用 make 也可以达到此目的。
 - ❑ **make clean**：删除之前编译时生成的可执行文件及目标文件（形如*.o 的中间文件）。
 - ❑ **make distclean**：除了删除可执行文件和目标文件以外，把 configure 所产生的 Makefile 文件也清除掉。通常在发布软件前执行该命令。
 - ❑ **make install**：将使用 make all 或 make 命令产生的可执行文件以软件的形式安装到系统中。若使用 bin_PROGRAMS 宏，程序将会被安装到 /usr/local/bin 下，否则安装到预定义的目录下。
 - ❑ **make dist**：将程序和相关的文档包装为一个压缩文档以供发布。执行完该命令，在当前目录下会产生一个名为 PACKAGE-VERSION.tar.gz 的文件。PACKAGE 和 VERSION 这两个参数是来自 configure.in 文件中的 AM_INIT_AUTOMAKE(PACKAGE, VERSION)。如在上个例子中执行 make dist 命令，会产生名为“hello-1.0.tar.gz”的文件。
 - ❑ **make distcheck**：与 make dist 类似，但是加入了检查包装以后的压缩文件是否正常。

➤ **归纳总结**

- 在存放源代码的目录下执行 autoscan 命令生成 configure.scan 文件。
- 将 configure.scan 文件改名为 configure.in，并对其默认配置进行修改。
- 执行 aclocal、autoconf 两个命令，分别生成 aclocal.m4、configure 文件。

- 创建一个名为 Makefile.am 的文件，输入相应内容。
- 执行 automake --add-missing，它根据 Makefile.am 文件，生成 Makefile.in。
- 执行 ./configure 脚本文件，它根据 Makefile.in 文件，生成最终的 Makefile 文件。

通过以上步骤，在源代码所在目录下自动生成了 Makefile 文件。

五、 版本管理

➤ 为什么要版本管理？

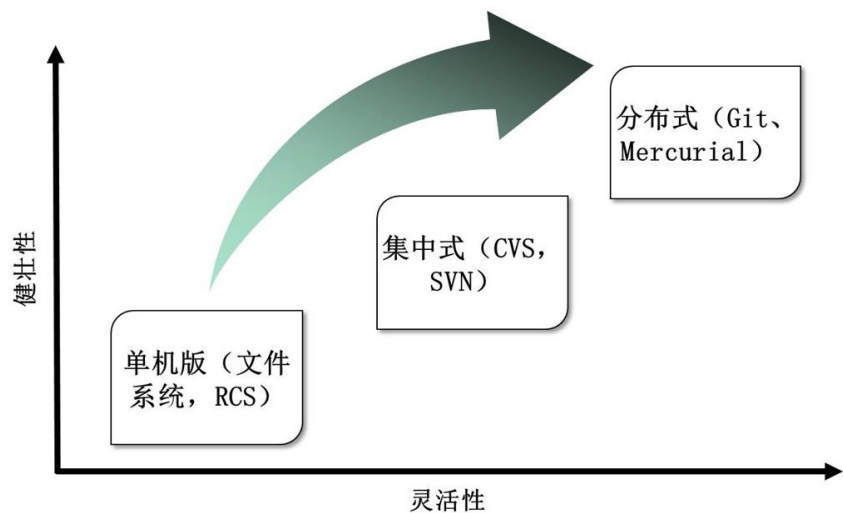
■ 团队协作

- ◆ 文件管理 历史追踪 冲突管理 版本发布

➤ 常见的版本管理软件

简 称	全 名
CVS	Concurrent Versions System
VSS	Micorosoft Visual SourceSafe
SVN	Subversion
Git	
TFS	Team Foundation Server
Mercurial	

版本管理发展过程

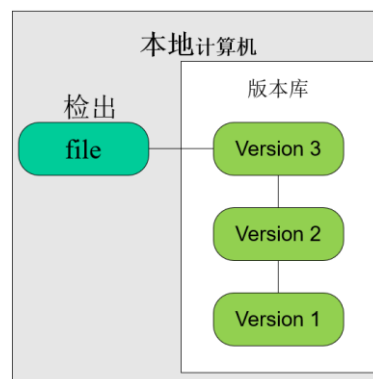


➤ 本地版本管理

- 版本库：个人电脑/服务器
- RCS:

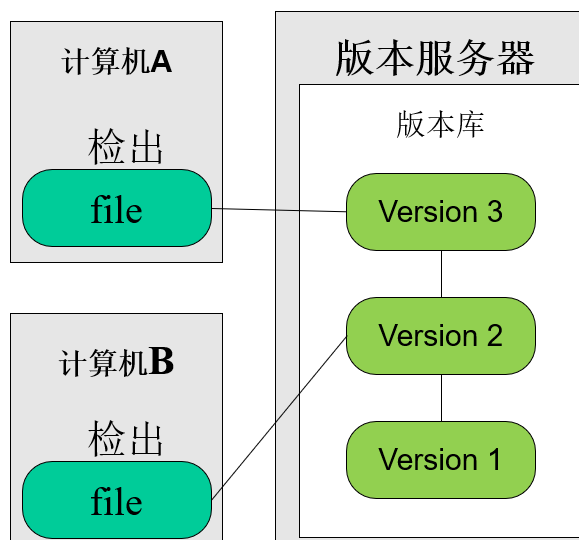
Revision Control System
可追踪修改历史

- 问题：如何协作？



➤ 集中式版本管理

- ◆ 版本库：版本服务器
- ◆ VCS:
 - Version Control System
 - 又称 CVCS (Central VCS)
 - 有 SVN、CVS、firefly 等产品
- ◆ 问题:
 - 服务器会停
 - 网络会不通



➤ 分布式版本管理

◆ 版本库

版本服务器

个人计算机

◆ VCS:

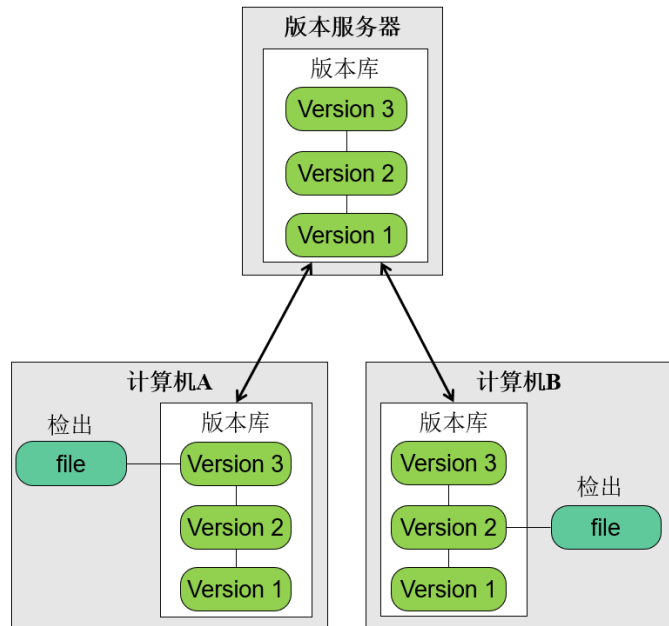
又称 DVCS (Distributed VCS)

有 GIT、HG、bazaar 等产品

◆ 问题:

乱成一团?

分支管理机制



➤ 常用术语

- **仓库(Repository):** 仓库就是当前的和历史的处于版本控制之下的文件所在的地方, 通常在服务器端。
- **工作版本 (Working copy):** 从档案库中取出一个本地端的复制, 所有在档案库中的档案更动, 都是从一个工作版本中修改而来的, 这也是这名称的由来。
- **分支(Branch):** 在一个时间点, 复制一份处于版本控制之下的文件, 从这之后, 这两份拷贝就可以独立的互不干扰的进行各自开发。
- **取出(Check-out):** 一次“取出”, 就是在本地创建一份仓库的工作拷贝。
- **提交(Commit):** 一次“提交”, 将本地的修改写回到仓库或合并到仓库。
- **冲突(Conflict):** 当开发者们同时提交对同一文件的修改, 而且版本系统不能把它们合并到一起, 就会引起冲突, 就需要人工来进行合并。
- **合并(Merge):** 合并就是把所有对文件的修改统一到文件里。

➤ GIT 设计目标

■ 满足大规模项目需要

- ◆ 快速 简单 完全分布 高效存储

➤ GIT 工作模式

◆ 版本库初始化

个人计算机从版本服务器同步

◆ 操作

90%以上的操作在个人计算机

添加文件

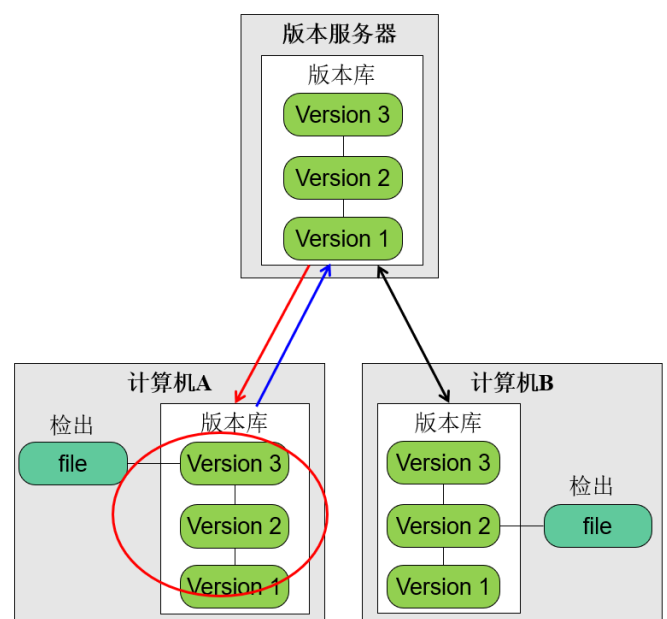
修改文件

提交变更

查看版本历史等

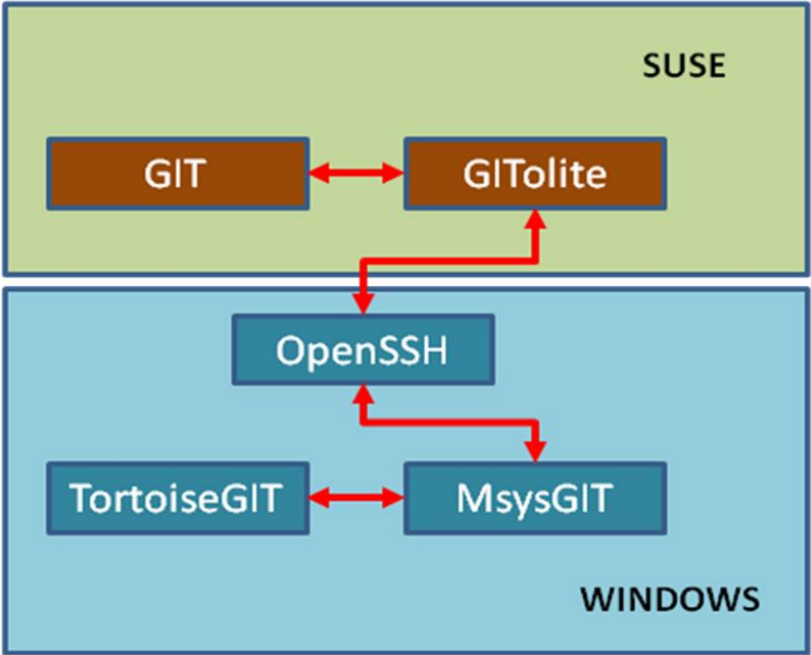
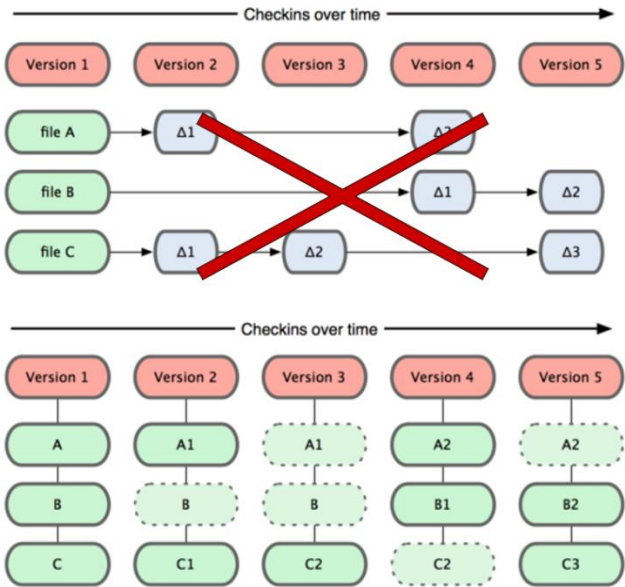
◆ 版本库同步

将本地修改传送到版本服务器

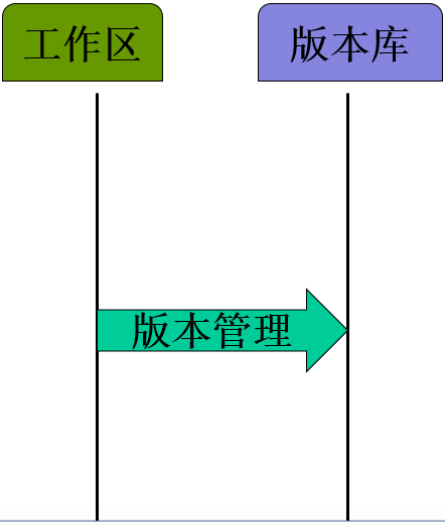


GIT文件存储

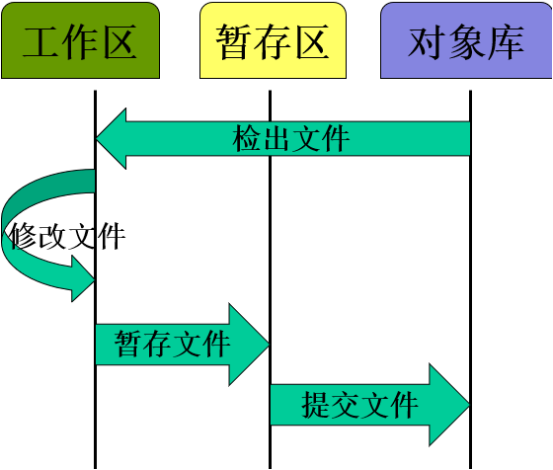
GIT版本库结构



- 文件状态 a
- 工作区
 - 工作目录: .git 文件夹所处目录
 - 文件状态
 - ◆ 未跟踪: 处于工作目录, 但未被版本库管理的文件
 - ◆ 已跟踪: 已被版本库管理的文件, 可能存在于工作目录, 也可能不存在于工作目录中



- GIT 文件状态
- GIT 文件
 - 已被版本库管理的文件
 - 已修改
 - ◆ 在工作目录修改 GIT 文件
 - 已暂存
 - ◆ 对已修改的文件执行 git 暂存操作, 将文件存入了暂存区
 - 已提交
 - 将已暂存的文件执行 git 提交操作, 将文件存入对象库



GIT 常用命令

- 获得版本库
 - git init/git clone
- 版本管理
 - git add/git rm/git commit
- 远程协作
 - git pull/git push
- 查看信息
 - git help
 - git log
 - git diff